Computers and Electrical Engineering 123 (2025) 110305

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

An effective IDS using CondenseNet and CoAtNet based approach for SDN-IoT environment

Dimmiti Srinivasa Rao, Ajith Jubilson Emerson *

School of Computer Science and Engineering, VIT-AP University, Amaravathi, Andhra Pradesh, 522237, India

ARTICLE INFO

Keywords:
CondenseNet
IoT
SDN
CoAtNet
CTGAN

ABSTRACT

A developing technology called the Internet of Things (IoT) allows smart objects to interact over various heterogeneous channels, whether wired or wireless. However, for traditional networks, effectively controlling and managing the data flows of many devices has become difficult. Software-defined networking (SDN) provides a solution to this problem. It has attempted to address several IoT issues, such as flexibility, diversity, and intricacy, because it is programmable, adaptable, fast, and provides a broad overview of the network. As a result, the system for attack detection and mitigation presented in this research leverages deep learning techniques to analyze SDN logs. Once the attack detection process has begun, the input data can be preprocessed using various techniques to replace missing values and prepare the data for additional processing. Subsequently, CondenseNet and Osprey Optimization Algorithm (OOA) are utilized for feature extraction and selection to identify more noteworthy characteristics. Lastly, the very accurate attack prediction is provided by the CoAtNet-based classifier, which is in charge of identifying intrusions. An efficient mitigation procedure was carried out to shield the network from attack after intrusion detection. Furthermore, a conditional tabular generative adversarial network is used to augment the data and correct class imbalance. To validate our proposed model, we conducted research and testing on InSDN, Bot-IoT, and IoT-23 datasets and achieved 99.74 %, 99.61 %, and 99.64 % accuracy, respectively. These experimental findings demonstrate that the suggested framework performs better than current state-of-the-art systems, achieving higher accuracy and a lower false alarm rate.

1. Introduction

An extensive network of devices connected by various communication interfaces is referred to as the Internet of Things (IoT). IoT devices stand out for having excellent mobility and broad coverage capabilities. However, these networks present several difficulties due to their complex structure, diverse channels of communication, and heterogeneous appearance [1–3]. These difficulties include balancing network traffic, maintaining quality of service (QoS), streamlining routing procedures, and preserving security precautions. Software-defined networking (SDN) emerges as a pivotal solution for effectively managing and controlling large, intricate, and diverse IoT networks [4,5]. Combining SDN and IoT, or SDN-IoT, represents a paradigm change in networking models.

In the SDN framework, the SDN controller offers centralized network intelligence and assists network managers in dynamically and programmatically configuring network traffic patterns to monitor, safeguard, and optimize network resources. In addition to

* Corresponding author.
E-mail addresses: srinivasa.21phd7126@vitap.ac.in (D.S. Rao), ajith.jubilson@vitap.ac.in (A.J. Emerson).

https://doi.org/10.1016/j.compeleceng.2025.110305
Received 23 August 2024; Received in revised form 7 March 2025; Accepted 22 March 2025
Available online 5 April 2025

managing the heterogeneous IoT, the SDN controller may keep an eye on all incoming and outgoing traffic [6–8]. Unfortunately, security attacks can target the centralized controller in an SDN-IoT configuration. The entire network is vulnerable to attack if the controller is compromised. Attackers can take control of the sensing layer (sensors) and data layer (switches) and manipulate them by creating fictitious traffic, turning off SDN-enabled switches for a predetermined amount of time, stealing confidential data, and erasing switch flow entries to reduce network performance [9–11]. For instance, the Mirai botnet malware compromises the default-configured IoT devices and starts command-and-control contact with the remote attacker server to join the bot army. These infected bots can flood the SDN controller with malicious network traffic. When controller or network resources are saturated, DDoS attacks may cause the entire network to go down. When the IoT devices are on the network, there are several additional attack options, such as topology poisoning, which compromises the controller or switches with known vulnerabilities in SDN. Consequently, an intrusion detection system (IDS) is necessary to protect the SDN controller by detecting and mitigating network attacks.

Conventional methods of network security and malicious activity interception, like data encryption, user authentication, and anti-malware software, are commonly employed and tailored to address specific security scenarios. Unfortunately, because of their rigidity and poorer ability to identify different types of attacks quickly, these security measures have shown to be ineffectual [12–14]. To address this, numerous academics have proposed models based on machine learning to limit the flow of attack traffic within the SDN-IoT network. They work effectively for low-dimensional attributes and small-sample data analysis, yet they are inappropriate for applications requiring large and high-dimension samples. Additionally, it necessitates manual feature engineering, which increases the efforts and duration required to select the attributes effectively.

On the contrary, the shortcomings of traditional machine learning are successfully addressed by deep learning. It can quickly and effectively process large amounts of high-dimensional data, learn features, and represent complex qualities as abstract data features [15–17]. Nevertheless, most deep learning-based anomaly detection methods rely on a single model, making it difficult to reliably detect abnormal activity in real time because the model may be unable to handle the varied and changing characteristics of network traffic and attack patterns. Also, a single model may fail to generalize across diverse types of attacks or anomalous behaviours, resulting in false positives or missed detections, limiting its ability to identify concerns in real-time effectively. To make matters worse, most current methods solely address attack detection; relatively few studies focus on protective measures. Moreover, the datasets that the current systems rely on are too inconsistent and outdated [18–20]. Unbalanced information is another significant issue that has not received much attention in earlier studies.

It motivates this research to use deep learning-based strong hybrid techniques to categorize and mitigate several SDN-IoT intrusions. The proposed framework consists of the following elements: Several preprocessing and Conditional Tabular Generative Adversarial Networks (CTGAN) based data augmentation are performed initially. Next, a CondenseNet-based technique extracts essential aspects of information from an SDN-enabled IoT network. Subsequently, the Osprey optimization algorithm (OOA) is added to pick more appropriate features and reduce network complexity. Next, a CoAtNet-based approach is used to examine and identify network intrusions. At last, the controller implements a successful mitigation scheme to return network activity to normal.

The significant contributions of this paper are,

- We used a data balancing strategy based on CTGAN to address the issue of low accuracy score and large false negative ratio resulting from imbalanced samples in the dataset.
- This work offers an efficient feature extraction and selection methodology that uses CondenseNet and the OOA approach instead of the conventional feature extraction strategy. This methodology helps to choose the most pertinent characteristics for IDS in SDN-IoT frameworks.
- A CoAtNet-based deep learning method is utilized to identify and categorize various network threats.
- This study presents an effective mitigation strategy for attack blocking and flow cleaning to efficiently reduce network intrusions.

The rest of the paper is structured as follows: Section 2 summarizes some existing research methods. The materials and techniques used in this work are described in Section 3, and the results and key discoveries are presented in Section 4. The limitations and future directions are described in Section 5, and the paper is concluded in Section 6.

2. Literature review

In SDN-based cloud systems, Songa and Karri [21] presented an integrated SDN framework that makes it possible to detect DDoS traffic patterns early. In this work, the relevant features were selected by Recursive Feature Elimination, and then the Density-Based Spatial Clustering technique formed the features into clusters according to timestamps. Afterwards, by releasing anomaly scores, each cluster was examined for any malicious traffic using time series algorithms. To categorize the traffic sample as either normal or DDoS, the rule-based classifier was utilized to calculate the sum of anomaly scores. Finally, the framework sounds an alarm and triggers the countermeasure portion when a sample is anomalous. Using the CICDDoS 2019 dataset, they assessed the suggested model using several performance criteria.

A hybrid Deep AutoEncoder with a Random Forest classifier model was developed by Mhamdi and Isa [22] to improve attack detection results in a native SDN context. An adaptive framework for threat mitigation is also incorporated in this paper. A three-layer defence mechanism was included in the suggested framework to identify and stop attacks. Entropy-based detection, proactive services monitoring in the application layer, and hybrid machine learning in the control layer serve as its foundation. Finally, the authors analyzed the efficacy of their approach on the CICIDS2017 dataset and obtained 98.16 % anomaly detection accuracy.

SDN-enabled hybrid-DL technique for intrusion detection in an IoT setting was created by Wahab et al. [23]. To find SDN

intrusions, they first pre-processed the data and then used the hybrid Bi-LSTM+GRU model to for threat identification. They used the CICDDoS2019 and N-BaIoT datasets to assess performance using a range of performance measures. Results were compared using the Cu-GRU+LSTM and Cu-GRU+DNN models on the same datasets.

Alasali and Dakkak [24] have developed a stacking classifier model for an SDN environment that consists of multiple base classifiers to provide a DDoS prediction model. This work contains three main modules: Attack Detection, Stacking Classifier, and Feature Creation. To determine the required characteristics for each network flow, the Feature Creation module first gathers network flows from the switches at predetermined intervals. After that, random forest based method was used to choose features. The DT, RF, LR, MLP, and KNN models are then used in the Stacking Classifier to process the chosen features. The Meta leaner then decides and detects the attacks in the attack detection module based on the output of the stacked classifier

For DDoS attack detection and mitigation on IoT networks, Khedr et al. [25] offer an SDN-based four-module system. The proposed frameworks consist of four main modules. The first module provides an early detection procedure that utilizes the average drop rate concept. The second section included a double-check mapping mechanism to aid in early data plane attack detection. Feature extraction, training and testing, classification, and data preprocessing were the four phases of the machine learning-based detection program that made up the third component. This module used seven characteristics to detect DDoS attacks. A procedure for mitigating attacks is introduced in the final module.

A machine learning-based method for identifying DDoS attacks in an SDN-WISE IoT controller has been proposed by Bhayo et al. [26]. To replicate the creation of DDoS attack traffic, they established a testbed environment and implemented a machine learning-based detection module into the controller. A logging mechanism was added to the SDN-WISE controller that records the traffic by writing network logs into a log file, which is then preprocessed and transformed into a dataset. Then, the SDN-IoT network packets were categorized by the machine learning DDoS detection module, which is integrated into the SDN-WISE controller, using the Support Vector Machine, Decision Tree, and Naive Bayes algorithms. Finally, they compared the outcomes produced by the machine learning DDoS detection module and assessed the effectiveness of the suggested framework using various traffic simulation situations.

To efficiently distinguish DDoS attacks, Polat et al. [27] delivered a multi-phase detection network that combined a decision tree and a 1D-CNN-based method. In this framework, 1D-CNN was used to retrieve in-depth characteristics, and a decision tree was used to identify the intrusions. The authors developed a personal dataset to train and evaluate this model. Based on the outcome, we found that the recommended model detected network attacks with 97.8 % accuracy.

El-Sayed et al. [28] presented the MP-GUARD framework to detect intrusions in SDN. This was accomplished using two fundamental modules: Multi-Pronged Intrusion Detection and Mitigation (MPIDM) and P4-Assisted Cooperative Traffic Monitoring (CTM-P4). Using the linked state tables in P4-enabled switches, CTM-P4 module enabled the dynamic feature extraction by facilitating real-time communication between various controllers. For thorough network analysis, this module presented a new feature set of 22 (12 extracted and 10 computed). After that, MPIDM uses CTM-P4′s comprehensive network insights to detect and mitigate attacks. This module presented Dynamic P4-Based Feature Selection with Stacked Ensemble Learning (SELDP4-FS) for attack detection and mitigation. For experimental analysis, ToN-IoT, Edge-IIoTset, and CICIoT2023 datasets were applied.

For SDN environments, Majidian et al. [29] presented intrusion detection based on random forests. It is divided into two stages. The network topology structure was separated into a number of subdomains during the first phase, and each subdomain was given a controller node. Then, in the second stage, feature ranking was done using techniques based on Least Squares (LS) and Information Gain (IG). The incursion in each subdomain was then detected using a machine learning model based on a random forest. To assess performance, they used the NSLKDD and NSW-NB15 datasets.

The SAnDet architecture was introduced by Zavrak and Iskefiyeli [30] for anomaly-based intrusion detection in SDN architecture. In this work, they employed the Replicator Neural Networks (RNN), a specific variation of the autoencoder, to extract the network properties and the EncDecAD technique, a unique kind of LSTM network that can identify network attacks. In the end, they counteract detected attacks or breaches by adding (or changing) flow entries to the flow table of the OF switch and adding malicious addresses to a blacklist database. For performance evaluation, they utilized the ISCX 2012 dataset.

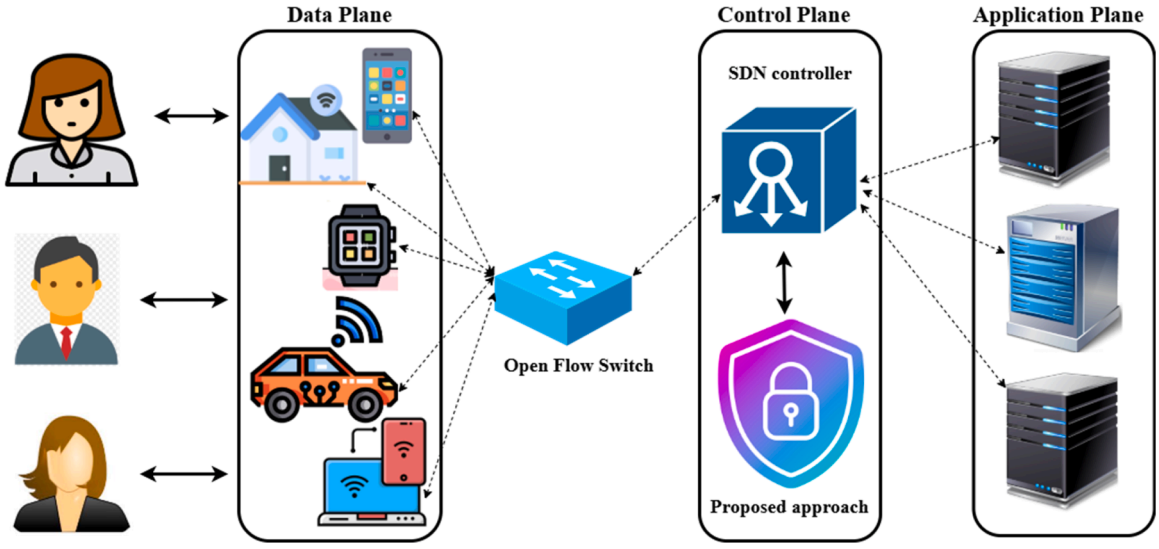
Upon reviewing the literature, we discovered that a variety of techniques have been successfully applied to intrusion categorization. Nevertheless, each approach has serious disadvantages, which are detailed in Table 1. This paper presents an efficient CondenseNet and CoAtNet-based method to address these problems. CondenseNet-based feature extraction is a good option for managing complicated datasets because it has a number of benefits. Due to its deep connection, feature maps from all previous layers can be reused, producing a rich and varied feature representation. It accomplishes a more petite model size and less computational complexity by using group convolutions and removing unnecessary connections, which makes it perfect for settings with limited resources. Additionally, incorporating OOA-based feature selection guarantees that only the most important and influential characteristics are utilized. Furthermore, CoAtNet's scalable and modular architecture facilitates hierarchical data processing and offers a strong foundation for contemporary classification problems. It is made to effectively manage computational and memory resources, guaranteeing good performance even with big datasets. It can manage a variety of datasets and tasks thanks to its generalization and robustness, which makes it especially well-suited for classifying SDN and IoT traffic. We chose more recent datasets (InSDN, Bot-IoT, and IoT-23) that are more appropriate for SDN setups for performance measurement. Additionally, we used a CTGAN-based augmentation technique to address imbalanced data problems in these datasets. Moreover, the suggested approach not only recognizes network attacks but also counteracts them, which is frequently overlooked in previous studies. Finally, the previously described benefits of our suggested approach strengthen its ability to identify network intrusions in SDN settings

Table 1
An overview of the literature review.

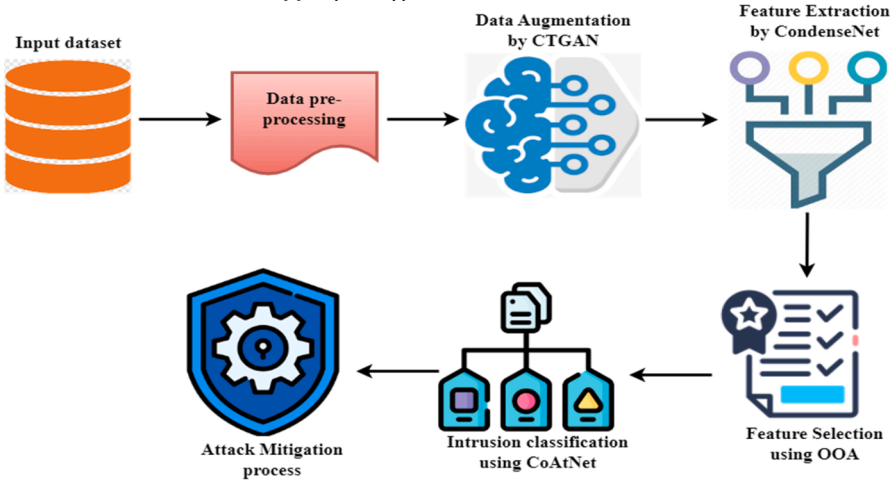
Paper	Methods	dataset	Evaluation criteria	Drawbacks
Songa and Karri [21]	Rule-based classifier	CICDDoS 2019	Accuracy, precision, recall and f1-score	This method's scalability is limited when used in large, complicated SDN settings. Time series analysis may bring latency into real-time detection and mitigation, especially in high-traffic settings. Furthermore, the framework's reliance on predefined thresholds and feature reduction procedures may reduce its adaptability to emerging attack plans in dynamic network environments.
Mhamdi and Isa [22]	Random forest	CICIDS2017	Accuracy, precision, recall and f1-score	The entropy-based detection is dependent on the network state, and alterations in structure or gadgets may necessitate retraining, creating delays and reducing performance. Moreover, the Proactive monitoring-based mitigation process consumes additional resources, slowing down the system and increasing costs. It may also be overly slow to mitigate quick or complex attacks since it is based on static rules.
Wahab et al. [23]	Bi-LSTM+GRU	CICDDoS2019 and N-BaIoT	Accuracy, precision, recall, f1-score, false discovery rate, false positive rate.	The imbalanced dataset used in this study may cause overfitting, in which the model primarily learns patterns from the primary class while ignoring cases from the smaller category
Alasali and Dakkak [24]	Stacked classifier	Personal dataset	Accuracy, precision, sensitivity, specificity and f1-score	Training several models is necessary for stacking classifiers, which raises training time and computing costs.
Khedr et al. [25]	BLR, GNB, SVM, kNN, DT, and RF	Edge-IIoTset dataset	accuracy, precision, F1-score, recall, specificity, average detection time, false-negative rate, false discovery rate, false-positive rate, and negative predictive value	Several detection and mitigation stages can cause network delays and congestion, particularly in high-traffic scenarios.
Bhayo et al. [26]	Support Vector Machine, Decision Tree, and Naive Bayes	Personal dataset	Accuracy and detection rate	This strategy did not include a various intrusion kinds or datasets. As such, the approach may not be generalizable to complex and unique intrusions.
Polat et al. [27]	Decision tree	Personal dataset	Accuracy, precision, recall, specificity and f1-score	Because of its dependence on a particular experimental dataset, the model can be overfit to its particular features, which would lead to poor performance on unknown attacks.
El-Sayed et al. [28]	Stacked Ensemble Learning	ToN-IoT, Edge-IIoTset, and CICIoT2023	Accuracy, precision, recall, specificity, NPV, MCC, FPR, FNR, FDR, and f1-score	Stacked Ensemble Learning with SELDP4-FS increases the computational burden and latency.
Majidian et al. [29]	Random forest	NSW-NB15 and NSLKDD	Accuracy, precision, recall, and f1-score	IG and LS based methods may fail to capture complex feature interactions, leading to suboptimal feature sets.
Zavrak and Iskefiyeli [30]	SAnDet	ISCX 2012	Accuracy, AUC	Outdated dataset. Relying solely on blocklisting harmful addresses would not be sufficient to counteract dynamic and quickly evolving attack techniques, like IP spoofing.

3. Proposed methodology

The suggested framework is intended for use in an SDN-based IoT context. Six modules make up this system: attack mitigation, anomaly detection, feature extraction, feature selection, data augmentation, and preprocessing. Fig. 1(a) presents the proposed approach in the SDN-IoT framework, and Fig. 1(b) displays the architecture of the proposed method. Using this method, the SDN controller keeps an eye out for network attack traffic abnormalities by monitoring each switch separately. SDN agents preprocess the data at switches to transform raw network flow traffic into normalized data. Next, to balance the attack classes in the dataset, CTGAN-based data augmentation is put into practice. The network features are then extracted using a CondenseNet-based technique. To improve process compatibility and lower computing complexity, OOA-relevant features are selected. To categorize the traffic sample as either normal or attack, the CoAtNet-based network is finally implemented. CondenseNet for feature extraction and CoAtNet for classification were chosen because of their complementary benefits in intrusion detection in SDN systems. Through feature condensation and dynamic channel selection, CondenseNet effectively gathers characteristics while lowering complexity, guaranteeing that only the most pertinent features are utilized, enhancing generalization, and minimizing overfitting. On the other hand, CoAtNet's multi-scale attention method, which enables it to concentrate on several levels of feature interactions, enhances attack categorization. As a consequence, it is better able to differentiate among various kinds of attacks and regular traffic, capturing both specific local characteristics and more general global circumstances. For real-time detection in SDN systems with restricted resources, our method strikes a balance between computational cost and performance. Compared to other deep learning architectures, the scalability, adaptability, and robustness of CondenseNet and CoAtNet are superior, which is essential for identifying a variety of threats while



(a) Proposed approach in SDN-IoT framework



(b) Overview of the proposed approach

Fig. 1. System architecture (a) proposed approach in SDN-IoT framework; (b) Overview of the proposed approach.

consuming the fewest resources possible. Finally, to safeguard network integrity and lessen the impact of attacks, the framework triggers the mitigation module when a sample is abnormal. The brief explanations of each module are provided below.

3.1. Preprocessing

Data preprocessing is used to clean up, normalize and prepare the data for further processing. It is a crucial step because the data may contain noise in the way of insignificant attributes, missing values, or variations that do not correspond to genuine patterns. Such noise might decrease the model's performance by generating inconsistencies that lead to underfitting or overfitting. Therefore, in our work, the following preprocessing steps are carried out.

- ***Handling Missing Values:*** It is one of the most important and initial steps in any data preparation strategy. Specific protocols, such as address resolution protocol (ARP), are missing values for the destination and source port numbers in our evaluation datasets. These missing data were purposefully adjusted to the incorrect port number (− 1) to resolve this problem. Moreover, the duplicate values are removed from the dataset.
- ***Handling Categorical Data:*** In our work, we used label encoding to translate the values of the categorical features into sequential numeric values. Every dataset is subjected to label encoding separately. For example, categorical columns (such as "Protocol", "stage", and "Attack Type") in the BoT-IoT or IoT-23 datasets are converted into integers that correspond to the categories. For instance, the categorical values like "REQ", "CON," and "RST" that are contained in the "state" property are encoded as "1, 2," and 3," respectively. Deep learning algorithms can handle processing and analysis more quickly due to this numerical representation.
- ***Data normalization:*** When features have varied scales, normalization is a crucial preprocessing step. Certain features with wider ranges may control the learning process and produce biased predictions if scaling is not done correctly. In order to guarantee that every feature contributes equally throughout the model training phase, we employed Min-Max Normalization to scale the features to a particular range, like [0, 1]. The following formula was used to carry out the normalizing process using the min-max transformation:

$$x_N = \frac{(x - x_{min}).(b - a)}{x_{max} - x_{min}} + a \tag{1}$$

In this case, x represents the initial value, and x_{min} and x_{max} are the dataset's lowest and maximum values. The new minimum and maximum values, which specify the range of interest, are represented by the variables a and b. This method makes sure that every feature is scaled from 0 to 1, which keeps any one characteristic from having an excessive impact on the model. To ensure that every feature contributes fairly, a feature with a wide range (such as 0 to 1000) is normalized to lie within [0, 1]. For example, Continuous features such as packet sizes, flow durations, etc., were normalized using Min-Max normalization.

3.2. Data augmentation using CTGAN

After the preprocessing, CTGAN [31]-based data augmentation is performed to tackle the unbalanced data problem and reduce network fitting. It removes certain instances from majority classes and increases the number of instances in minority classes. The main benefit of employing CTGAN is that it can produce realistic synthetic samples for attack classes that are underrepresented. It immediately enhances the model's capacity to identify minority class attacks that would otherwise go unnoticed because of data imbalance. CTGAN enables the model to learn a variety of features and patterns that may not be adequately represented in the original dataset by creating new samples that closely mimic actual attack data. In terms of the effect on detection accuracy, we found that adding augmented data from CTGAN enhanced the model's functionality, particularly for underrepresented attack types. As a result, the model was able to identify a wider variety of attack patterns, which improved detection rates and overall classification outcomes. It is used only during the training stage to produce simulated data to aid the model to generalize more effectively. During the testing stage, the model uses only genuine, unaugmented data for prediction because the purpose is to evaluate the model's performance on actual network traffic.

CTGAN is a GAN-based framework designed for the synthesis of tabular data. The main objective of CTGAN's functionality is to lessen the difficulties associated with tabular data modelling utilizing GAN architecture. More precisely, the CTGAN strategy manages non-Gaussian and multimodal distributions by using mode-specific normalization (MSN), which converts continuous values of any distribution into a limited vector. For neural networks, this illustration is suitable.

The two neural network stages that comprise CTGAN are a generator and a discriminator. CTGAN uses mode-specific normalization to handle the non-Gaussian and multimodal variation of successive fields in tabular data. To solve the issue of unbalanced collections in continuous columns, sample-based training and a conditional generator are used. Additionally, CTGAN utilizes a few of the most recent advancements in GAN training, such as the WGAN-GP's loss function and the crucial design of PacGAN, to enhance the standard of created data and training consistency. The CTGAN loss function is shown in the following Equation.

$$L = E_{G(z)\sim P_g}[D(G(z))] - E_{x\sim P_r}[D(x)] + \lambda E_{y\sim P_y}\left[\left(\parallel \nabla_y D(y) \parallel -1\right)^2\right] \tag{2}$$

In this instance, the gradient penalty factor's value is denoted by λ, and the sample y is constantly interpolated to the actual data x.

The probability distribution of generated data is denoted as P_g, the probability distribution of the real data is indicated by P_r and the interpolated data distribution is indicated by P_y.

The tabular data (T) provides random noise (z) and conditioning information (y) to the generator in CTGAN. The conditioning information might specify the kind of instances in each column, among other features of the resulting data. The generator then creates fictitious samples, which are forwarded to the discriminator (D) for a conclusion. The discriminator delivers a signal to the generator after determining via backpropagation whether the instance is authentic or fraudulent. This indication is used by the generator to modify its weights and improve its ability to produce reliable data. The newly generated data list is given in Appendix A, and the system architecture of CTGAN is given in Fig. 2.

3.3. CondenseNet-based feature extraction

Then, we use a CondenseNet-based deep learning approach to extract network features from the input data. A convolutional neural network is the foundation of CondenseNet, a densely linked network. Based on the widespread and apparent feature reuse structure of DenseNet, the lightweight network CondenseNet was developed. The CondenseNet network's superior feature extraction capability and increased computing efficiency allowed us to utilize it to extract network properties from internet traffic.

During the feature extraction process, the input is sent via a sequence of densely connected blocks, with each layer receiving feature maps from all previous layers. This deep connection allows the model to reuse data at several levels, collecting both fine-grained (low-level) elements like packet flow characteristics, gradients, and textures and abstract (high-level) features like attack signatures and behavioural patterns. Within these blocks, CondenseNet uses group convolution, which divides feature maps into smaller groups and applies convolution to each group separately. This strategy dramatically reduces computational complexity while maintaining the richness of feature representations. Furthermore, the permute operation dynamically rearranges the feature channels according to their relevance, ensuring that the most important characteristics are positioned for efficient processing. Learnable masks guide this reordering, ranking the paths based on their effectiveness in the task. CondenseNet improves feature-subset interaction by interleaving permuted channels across groups, enabling the model to acquire more varied and valuable descriptions. The more detailed layer-by-layer process is explained below.

In this network, several dense blocks are used to extract features. Two 1×1 LG-Conv (Learning Group Convolution) layers and one 3×3 G-Conv (Group Convolution) layer make up each block. The objective of these layers is to extract and enhance the features from the input data. Next, in order to stabilize training and enhance convergence, batch normalization is applied to the LG-Conv output. Next, non-linearity is added, and the representational power of the model is improved by applying ReLU activation. A permute operation is used in each 1×1 LGConv layer for channel shuffling in order to improve the efficacy and efficiency of feature learning. Additionally, this network's input and output neurons are separated into separate groups of g. As a result, the network may concatenate and execute regular spatial convolution over each independent group.

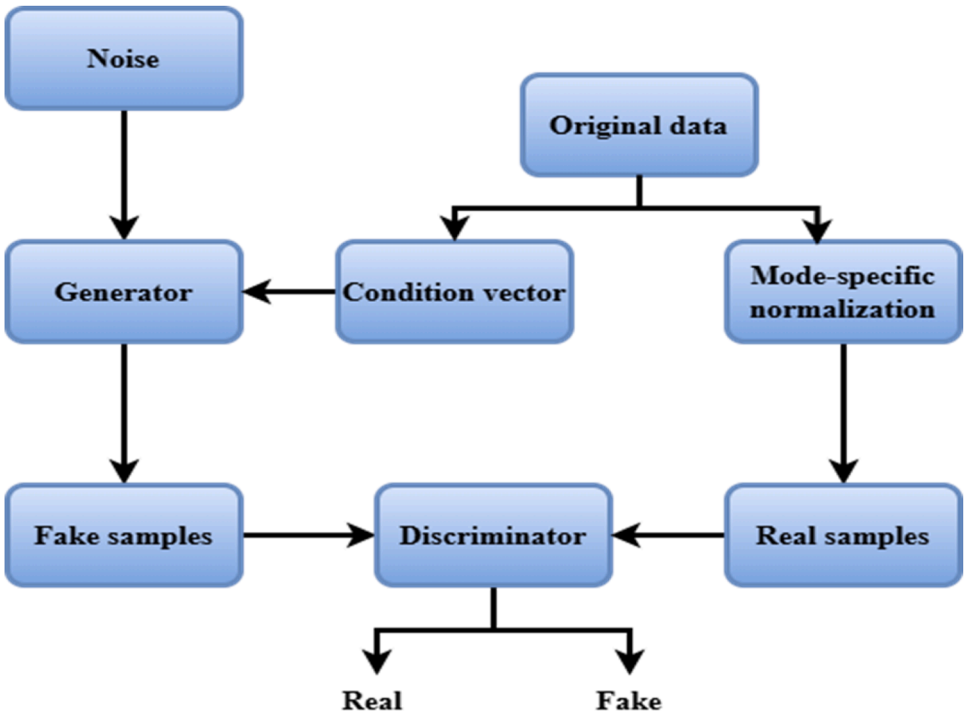


Fig. 2. CTGAN.

In this study, we substitute channel concatenation for element-wise addition because it can increase the channel and obtain higher performance by utilizing local and global feature maps. For a given layer l, the result of the LG-Conv operation combined with channel concatenation can be written as follows:

$$x_l = \mathrm{ReLU}(BN(LGConv\ ([x_0, x_1, x_2, \ldots, x_{l-1}]))) \tag{3}$$

Here, x_l is the output feature map of the L-th layer, and the concatenation of the feature maps produced by layers 0 to $l-l$ layers is denoted by $[x_0, x_1, x_2, \ldots, x_{l-1}]$, rectified linear unit activation function is denoted by ReLU, batch normalization denoted by BN and learning group convolution operation is denoted by LGConv.

One way to lessen the negative consequences of introducing 1×1 LG-Conv is to enable channel switching through the Permute layer. In addition, the condense factor is set to 4 to improve the computing efficiency of the network. Additionally, the original CondenseNet has an average pooling layer containing transition layers that separate blocks. Pooling layers were not required for the extraction of intrusion detection features. Therefore, we have skipped these transition layers in our model. The feature representation of the input data within the dense block is the output of the final layer in that block. These feature representations can be used for intrusion detection to identify pertinent patterns and traits that point to malicious or typical network behaviour. The architecture of CondenseNet is given in Fig. 3.

3.4. Feature selection using osprey optimization algorithm

From the extracted features, more significant features are selected using the Osprey Optimization Algorithm (OOA) [32]. We choose this approach because it effectively strikes a balance between exploration and exploitation, assisting in the identification of the most pertinent features while avoiding local optima. Furthermore, OOA provides lower computing costs and effectively manages

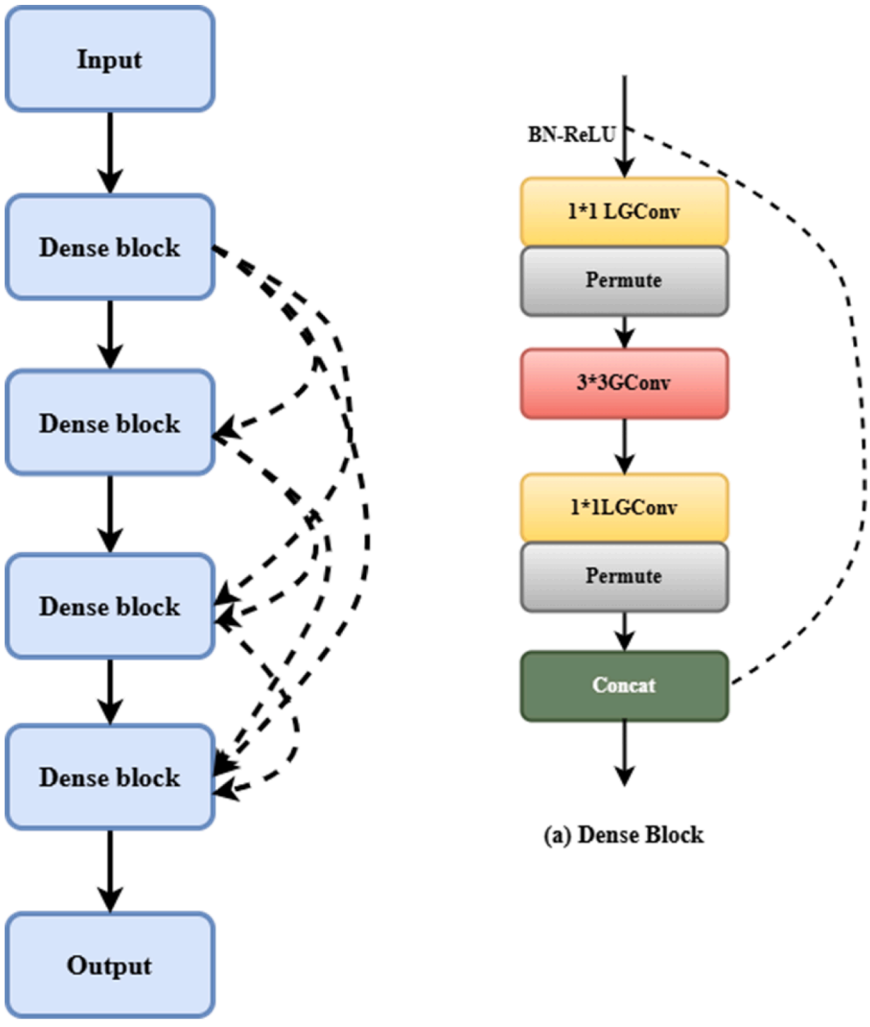


Fig. 3. CondenseNet.

high-dimensional data, which makes it perfect for enhancing classification performance in SDN-based intrusion detection systems.

This method is also called the Water Hawk algorithm. It is a simulation that solves complicated issues by mimicking the actions of ospreys. It is developed using the eating and hunting activities of ospreys. The OOA method is created using a mathematical formation of osprey intelligence.

3.4.1. Start of initialization

This algorithm is a population-based approach that goes through several rounds to find the optimal answer in a problem-solving space. Eq. (4) treats the Osprey as a network feature in the population, signified by a matrix. Every Osprey contributes information about its place to the problem-solving process, serving as a sample of the population.

$$O = \begin{bmatrix} O_1 \\ \vdots \\ O_i \\ \vdots \\ O_n \end{bmatrix}_{n\times m} = \begin{bmatrix} o_{1,1} & \cdots & o_{1,j} & \cdots & o_{1,m} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ o_{i,1} & \cdots & o_{i,j} & \cdots & o_{i,m} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ o_{n,1} & \cdots & o_{n,j} & \cdots & o_{n,m} \end{bmatrix}_{n\times m} \tag{4}$$

Eq. (5) is used to randomly initialize the 'n' features in the network.

$$O_{i,j} = LB + r_{i,j} \cdot \left(upb_j - lob_j\right) j = 1, 2, \ldots, m, \; i = 1, 2, \ldots, n \tag{5}$$

Here, lob_j and $\text{upb}_j$are the lower and upper bounds of the jth problem variable and$r_{\text{i,j}}$ indicates a random value among 1 and 0. The osprey population matrix is represented by the symbol "O." Additionally, O_i denotes the ith Osprey and$O_{i,j}$ denotes the ith Osprey in the jth dimension, and m and n are the counts of problem variables and the entire amount of ospreys. After problem evaluation, the objective function produces a set of values (Osprey). Eq. (6) uses a vector to show the problem's predicted values.

$$F = \begin{bmatrix} F_1 \\ \vdots \\ F_i \\ \vdots \\ F_n \end{bmatrix}_{n\times\ 1} = \begin{bmatrix} F(O_1) \\ \vdots \\ F(O_i) \\ \vdots \\ F(O_n) \end{bmatrix}_{n\times\ 1} \tag{6}$$

In this case, the collection of values for the objective function is denoted by F, andi^{th} the osprey value is indicated byF_i. The suggested solution's quality is assessed using the objective function of projected results. To select the ideal and weakest members, the iteration method considers the highest and lowest values. Following each cycle, the Osprey place is assessed and revised.

3.4.2. Exploration stage

Ospreys can detect and track fish underwater because of their superior eyesight. Ospreys locate their prey and then plunge into the water to pursue them. In the early phases, the population is revised based on osprey behaviour. The exploratory capacity of the OOA to find and avoid local optima is improved by mimicking the Osprey attack on prey, which changes the Osprey's location in the search space.

Each Osprey's superior objective value of undersea fish is considered by the location of other ospreys in the search area. The following equation expresses a set of prey for every Osprey.

$$Fpos_i = \{O_\omega | \omega \in \{1, 2, \ldots, n\} \wedge FH_\omega < FH_i\} \cup \{O_{best}\} \tag{7}$$

In this case, O_{best} denotes the finest Osprey, and the fish set for ith osprey is denoted by $Fpos_i$

The Osprey hunts underwater fish by randomly locating itself. Eq. (9) calculates its newest spot in the search area based on its progression in the route of the fish. Eq. (10) should be utilized to substitute the Osprey's last position if a new measurement of its location surpasses the prior objective value.

$$o_{i,j}^{pos1} = 0_{i,j} + ran_{i,j} \cdot \left(SF_{i,j} - R_{i,j} \cdot y_{i,j}\right) \tag{8}$$

$$o_{i,j}^{pos1} = \begin{cases} o_{i,j}^{pos1}, \; \text{lob}_j \le o_{i,j}^{pos1} \le upb_j; \\ \text{lob}_j, o_{i,j}^{pos1} < lob_j; \\ \text{upb}_j, o_{i,j}^{pos1} > upb_j; \end{cases} \tag{9}$$

$$O_{\text{i}} = \begin{cases} O_i^{pos1}, FH_i^{pos1} < FH_i; \\ O_i, else, \end{cases} \tag{10}$$

Here, $R_{i,j}$ denotes the random integer between 2 and 1, $ran_{i,j}$ defines the random interval [0,1], SF_isignifies the fish that the i^{th}Osprey has chosen, $SF_{i,j}isj^{th}$ a dimension, the objective function value denoted byFH_i^{pos1}, O_i^{pos1} denotes the ith Osprey's new spot according to the OOA first phase, $o_{i,j}^{pos1}$represents the jth dimension.

3.4.3. Exploitation stage

When the hunting is done, the Osprey will be in a secure location to consume fish. The Osprey's location in the search space is updated when it transports fish to an adequate spot to feed. It makes the OOA more capable of improving solutions and making use of neighbourhood searches.

By simulating osprey behaviour, Eq. (12) establishes the random position in the population in which every Osprey will consume fish that was hunted during the OOA exploration stage. Based on the objective function's best value obtained in Eq. (13), the Osprey's previous location is upgraded.

$$o_{i,j}^{pos2} = o_{i,j} + \frac{lob_j + ran_{i,j} \cdot \left(\mathrm{upb}_j - lo\omega_j\right)}{k},\ k = 1,2,\ldots,T,\ j = 1,2,\ldots,m,\ i = 1,2,3,\ldots,n \tag{11}$$

$$x_{i,j}^{pos2} = \begin{cases} o_{i,j}^{pos2},\ lob_j \le o_{i,j}^{pos2} \le upb_j; \\ lob_j, o_{i,j}^{pos2} < lob_j; \\ upb_j, o_{i,j}^{pos2} > upb_j; \end{cases} \tag{12}$$

$$o_i = \begin{cases} o_i^{pos2},\ \mathrm{FH}_i^{\mathrm{pos2}} < F\mathrm{H}_i; \\ o_i\ , else, \end{cases} \tag{13}$$

In this case, the number of repetitions is indicated by k, ran_{ij} denoted the random interval [0, 1], the objective function value is indicated by $\mathrm{FH}_i^{\mathrm{pos2}}$, the ith Osprey's new position according to the OOA second stage is indicated by o_i^{pos2}, and $o_{i,j}^{pos2}$ represents the jth dimension. The Osprey's position is calculated and compared to its previous value with each iteration of the objective function. If the new position provides a higher fitness value, it is adjusted accordingly. The optimal features from the dataset are then chosen based on the fitness function results. This paper assesses performance using the BoT-IoT, InSDN, and IoT-23 datasets, which comprise 44, 51, and 22 features, respectively. From those features, 10 (BoT-IoT), 20 (InSDN), and 10(IoT-23) optimal features are selected using the OOA algorithm, which is listed in Appendix B.

3.5. Intrusion classification using CoAtNet

After receiving the selected features, the CoAtNet uses them to categorize incursions. By layering convolution and attention vertically, CoAtNet integrates transformers with convolutional neural networks. This hybridization is mainly responsible for its ability to handle increased network traffic and scale with higher data quantities. Through multi-scale attention, CoAtNet's Transformer component efficiently manages global dependencies as network traffic grows. It enables the model to adaptively focus on various input data regions, detecting both short-term and long-term relationships. This attention method keeps the model performing well in a variety of traffic conditions and dynamically adapts to changing feature levels, which improves the model's scalability to increasingly complex datasets. Additionally, CoAtNet's architecture uses parallel processing to handle big datasets well without noticeably sacrificing accuracy or speed.

CoAtNet's inductive biases enable it to generalize similarly to ConvNets. CoAtNet's efficiency is further enhanced by faster convergence and improved transformer scalability. An MBConv block and self-attention are combined with the relative position in the CoAtNet model. Three qualities are achieved by this combination: translation equivariance, input-adaptive weighting, and global receptive field. The five stages (L0, L1, L2, L3, L4) of the CoAtNet design are essentially made up of the Conv2d, MBConv, MBConv, TFMRel, and TFMRel layers, correspondingly. In this, TFMRel is a Transformer block made up of an FNN layer after an MSA with relative position encoding, and Conv2d is a 2D convolutional layer. Channel D from L1 to L4 is doubled, and each level is repeated T times. For every Conv2d and MBConv block, the kernel size is 3; for every Transformer layer, each attention head's size is 32. The initial MBconv block performs a deep convolution to attain the interplay among the characteristics. Convolution for each channel is carried out independently via depthwise convolution, which may be stated as:

$$y_i = \sum_{j \in L(i)} w_{i-j} \circ x_j \tag{14}$$

Here, (i) indicates an ith local neighbourhood, and the output and input at the ith position are indicated by

$$x_i, y_i \in R^D \tag{15}$$

For the MBConv and Transformer blocks, pre-activation is used, where normalization takes place prior to layer operation, which makes it possible to establish the residual connection as follows:

$$x \Leftarrow x + Layer(Norm(x)) \tag{16}$$

Here, layer stands for MBConv, FFN, or self-attention layers. Norm stands for batch normalization for MBConv blocks and layer normalization for FFN and self-attention layers. Separate downsampling is done for the identification branch and the residual branch for the first block in each level from L1 to L4. Specifically, the max-pooling of stride two is applied to the input states of both branches for every Transformer block. A channel projection to the identification branch is also carried out to improve the hidden size. In

CoAtNet, we include a fully connected (dense) layer after the sequence of convolutional and attention layers. After this, a dropout layer is positioned to stop overfitting. Lastly, a softmax activation function-equipped output layer predicts the network anomalies. The system architecture of CoAtNet is given in Fig. 4.

3.6. Mitigation module

Once the attack is identified, the proposed mitigation module is activated. It must locate the malicious end host (attacker) and stop communication between the attacker and the victim. The suggested mitigation strategy is very different from conventional methods in a number of important ways. The mitigation of known vulnerabilities in existing IDS for SDN setups frequently depends on static rules or predefined blocklists. Because they might not react well to altered or unfamiliar attack patterns, these systems typically lack real-time adaptability.

On the other hand, our suggested approach continuously monitors network traffic and is dynamic and real-time. It can detect anomalous activities and more successfully neutralize threats by utilizing a combination of flow-level analysis and packet-level feature extraction. The controller can continuously update its knowledge of the network status because of the system's use of real-time data collection from the SDN switch. This real-time interaction guarantees that the system can react to new, developing attack vectors by instantly blocklisting suspicious packets and flows, in contrast to typical static mitigation techniques. By swiftly separating dangerous components from regular traffic, this proactive technique improves network security and increases the system's effectiveness in lessening the impact of attacks.

Creating flow controls to stop malicious traffic from advancing is the basic mitigation idea. We have combined it with a traceback method to improve its efficacy. We attempted to trace back traffic as closely as feasible to its source using the flow rules set forth by the network switches. It entails checking to see if the flow rules already loaded on the device address the necessary flow or not. If it happens, proceed to the device that is directly the cause of the installation of this flow rule and carry on in this manner until we reach the end host. One immediate benefit of backtracking is that it stops traffic at the server end. In the event of an attack, it enables us to redirect traffic to the immediate switch, which prevents load and bottleneck on other network segments. We may also avoid spoofing IP addresses by basing the traceback on the flow rule. Because spoof IP addresses are nonexistent or do not correspond to an attacker, they are employed for mitigation purposes.

To deal with each server being watched over or protected against an attack, the module initiates a thread during initialization. Connecting the switch to the server is now the first step in the mitigation procedure. The switches installed flow entries are read by the module. After that, a traffic selector is defined to choose which traffic is sent to the victim server. It compares the flow entries in the database with this traffic. Among all the flow entries that match, the one with the highest packet count is chosen. Next, the module takes flow entry fields and extracts incoming ports. The identity of the connecting device—a switch or an end host—is verified at this port. The module creates a flow rule to stop traffic from an end host if one is discovered. If a switch is found, the mitigation process described above is repeated using the identified switch as the base. It continues until the TTL expires or the end host is located. We can check for numerous servers at once because of the threaded structure, which shields additional servers from the attack. Algorithm 1 presents the module's pseudo-code.

4. Result and discussion

The outcomes of experiments conducted using the suggested architecture are covered in this section. This study offers a comprehensive assessment using three distinct and extensive datasets: Bot-Iot, InSDN, and IoT-23. We randomly split the dataset into testing (20 %) and training (80 %) for evaluation purposes. The following parameters are initialized for training deep learning networks: 0.9 for dropout rate, 8 for batch size, 0.8 for momentum, 1e-06 for weight decay rate, 0.001 for learning rate, and 100 for the number of epochs. These values were established empirically using the random search method, which investigates various combinations of hyperparameter values by randomly selecting samples from predetermined ranges. With this method, we were able to more quickly and empirically identify the ideal parameters.

This test was conducted in a Mininet setting, with RYU chosen as the controller and CC-Guard installed in the controller. The hosts are h1 to h10, the switches are S1 to S9, and the controllers are C1 to C3. Ubuntu 20.0.1, 16 GB of RAM, and an Intel Corei7–10,510 u CPU are the hardware and operating system configurations. The Keras framework is used to implement the suggested hybrid model.

4.1. Overview of the dataset

BoT-IoT dataset: It includes botnet and ordinary traffic in formats like CSV, argus, and PCAP files. It is publically available in https://research.unsw.edu.au/projects/bot-iot-dataset. It was produced by the Cyber Range Lab at The Center of UNSW Canberra Cyber. It simulates a practical network setting. The entire collection has >72 million records in it. A five percent subset of the dataset was used for our tests. There are 29 attributes in this dataset, which includes several attack types such as data filtration, Keylogging, Service Scan, DoS, and DDoS attacks.

InSDN dataset: In 2020, the InSDN dataset was released which is publically available in https://www.kaggle.com/datasets/badcodebuilder/insdn-dataset. Real-world network activity from an SDN setting was gathered by the researchers, who then categorized it according to the prevalence of DDoS attacks. In contrast to NSL-KDD and KDD99, InSDN offers an accurate depiction of traffic inside an SDN context. It was built with the CICFlowMeter tool, which produces a CSV file that includes >80 statistical features. There are 361,317 observations of attacks and normal traffic in the InSDN dataset, 292,893 observations of attacks, and 68,424 observations

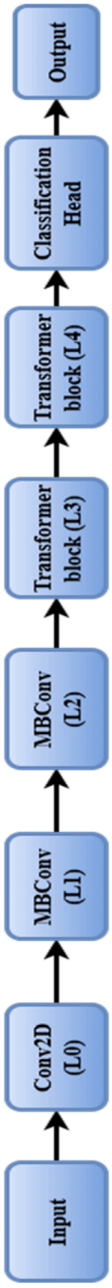


Fig. 4. CoAtNet.

Algorithm 1
Assign necessary constraints.

```
class TrafficBlocker
    procedure traceAndBlockTraffic(ip_Address, mac_Address, deviceId)
    Set maxBytes to 0
    Set targetPort to any port
    Get all flow entries for deviceID
    for each flowEntry ∈ flowEntries do
    traffic_Selector = defineTraffic_Selector(ip_Address, mac_Address)
    if traffic_Selector == flowEntry.selector() then
    if flowEntry.bytes() > max_bytes then
    Update maxBytes to flow entry's byte count
    Set targetPort to flow entry's port
    end if
    end if
    end for
    Get all connected hosts for deviceID
    for each host ∈ hosts do
    if IsHostAtPort(targetPort, host, deviceId,) == True then
    setFlowRule(targetPort, ip_Address, mac_Address, deviceId)
    end if
    end for
    Create a connection point for deviceID and targetPort
    Get all links connected to this point
    for each link ∈ links do
    srcDeviceId = link.src().deviceId()
    traceAndBlockTraffic (ip_Address, mac_Address, srcDeviceId)
    end for
    end procedure
    procedure createFlowRule(targetPort, ip_Address, mac_Address, deviceId)
    traffic_Selector=getTrafficSelector(targetPort, ip_Address, mac_Address, deviceId)
    traffic_Treatment = getTrafficTreatment(DROP)
    CreateAndInstallFlowRule(traffic_Treatment, traffic_Selector)
    end procedure
    end class
```

of regular traffic. It includes modern, well-known attack types like web exploitation, password guessing, DDoS, botnet, and probe attacks. Additionally, common apps like email, file transfer protocol (FTP), secure shell (SSH), distribution database system (DNS), and hypertext transfer protocol secure (HTTPS) are included in the dataset's normal network traffic.

IoT-23 dataset: This dataset was developed in the Avast AIC laboratory and publically available in https://www.stratosphereips.org/datasets-iot23. In January 2020, it was gathered in 23 various circumstances between 2018 and 2019—and was made accessible. Network packets from IoT devices provide this large dataset. After the network packets are retrieved, clean, malicious malware-infected labelled records are produced. Overall, there are 325,307,990 records in this set. The eighteen features in each record are divided into four groups: fundamental, stable, content, and time attributes. The IoT-23 dataset has five static attributes: destination and source IP, destination port, protocol, and source port. Packets in the same or different flows are used to extract additional characteristics.

4.2. Performance metrics

The problem of distinguishing malicious traffic from legal traffic is addressed in this study. The following metrics should be taken into consideration when assessing a detection strategy's effectiveness:

$$Accuracy = \frac{TrPv + TrNv}{TrPv + TrNv + FlNv} \times 100 \tag{17}$$

$$precision = \frac{TrPv}{TrPv + FlPv} \times 100 \tag{18}$$

$$Sensitivity = \frac{TrPv}{TrPv + FlNv} \times 100 \tag{19}$$

$$Specificity = \frac{TrNv}{TrNv + FlPv} \times 100 \tag{20}$$

$$F1 - score = 2\left[\frac{precision \times Sensitivity}{precision + Sensitivity}\right] \tag{21}$$

$$FalseAlarmRate = \frac{FlPv}{TrNv + FlPv} \times 100 \quad (22)$$

The following terms are used in the above equations: True Positive (TrPv) shows the number of malicious traffic that has been correctly identified as malicious traffic, True Negative (TrNv) shows the amount of ordinary traffic that is appropriately categorized as ordinary traffic, and False Positive (FlPv) reveals the amount of regular traffic that is mistakenly identified as malicious traffic. False Negative (FlNv) shows the number of malicious sites mistakenly identified as regular traffic.

4.3. Performance assessment on the InSDN dataset

Table 2 displays the evaluation standards used for our proposed model and the outcome of the multiclass classification of the InSDN dataset. Table 2 presents the results, which indicate that the suggested strategy effectively reached above 99 % accuracy for all the attack types using the InSDN dataset. What is interesting is that classifying DDos and Dos attacks produces relatively few false positives; on the other hand, distinguishing between regular and U2R attacks sometimes produces false positives since U2R attacks share characteristics with other members of the same class. Because of this, the accuracy of the U2R class is marginally lower than that of the other categories. Nevertheless, it has no effect on the suggested approach's overall effectiveness.

Table 3 and Fig. 5 show how our methodology compares with current practices. With very few features, our methodology achieves an impressive accuracy of 99.74 %. It was accomplished following the deployment of our unique feature selection process, which successfully whittled down the feature set from the initial 84 characteristics included in the InSDN dataset to the 11 most valuable features. One crucial component that contributes to our IDS's remarkable efficiency in processing and categorizing data is the decrease in feature space. Because of this, our methodology is appropriate for wireless networks, especially in scenarios with scarce resources like the IoT.

The Multilayer Perceptron (MLP) technique performs the lowest (97.3 % F1-score) when compared to all other methods due to its inability to handle imbalanced datasets, which causes the model to perform poorly on the minority class (intrusions) and skewed towards the majority class (regular traffic). The CTGAN-based data augmentation method used in our suggested approach boosted the samples in the minority classes, improved dataset balance, and produced results including 99.74 % accuracy, 99.72 % precision, 99.68 % sensitivity, 99.71 % specificity, and 99.70 % f1-score.

The classification outcomes demonstrate the superiority of the presented framework in the confusion matrix (Fig. 6). There are eight classes overall, as indicated by the square confusion matrix. The score on the diagonal shows the number of genuine optimistic predictions for each class. The matrix also offers information on incorrect classifications; for example, 34 cases of passwords anticipated to be DDoS and 44 cases of U2R predicted to be normal are examples of misclassifications.

We examined the Receiver Operating Characteristic (ROC) graph in order to confirm the effectiveness of the suggested method even more. To compare the false and true positive rates, the visualized results are plotted on the ROC curve. The ROC indicates the performance of multiclass classification problems, which is the main indication of the degree of separability. The presented model's ROC graph is shown in Fig. 7, indicating its superior performance across all attack classes. Furthermore, AUC values of >0.99 are found for both the attack and regular classes, suggesting that our suggested methodology is effective in detecting attacks in SDN-IoT setups.

4.4. Performance assessment on Bot-IoT dataset

First, let us look at the multi-classification outcomes. Initially, our goal was to analyze how our solution handled the intrusion detection issue. The performance results of the suggested technique using the One vs. another classes for multi-classification tasks are presented in Table 4. It is important to note that the method works well in the majority of classes, particularly the Normal and DDoS classes (99.93 % and 99.91 %, respectively). This outcome implies that, after accounting for both false positives and false negatives, the algorithm successfully classified cases from the "Normal" class with sufficient accuracy. Additionally, the "DoS" class has a high F1 score of 99.80 %, which suggests that recall and precision are well-balanced. For example, the algorithm produces accurate predictions in this class while striking a solid balance between false negatives and positives.

Following the class-wise prediction, we contrasted our findings with other research projects utilizing Bot-IoT data sets. The total outcome of the suggested model is contrasted with the dataset's current methods in Table 5. The outcomes demonstrate that the presented network delivers greater detection accuracy when compared to the current methods. In terms of F1-score, specificity,

Table 2
Attack class comparison on the InSDN dataset (%).

Classes	Specificity ↑	F1-score ↑	Sensitivity ↑	Accuracy ↑	Precision ↑
Web-attack	99.33	99.32	99.3	99.36	99.34
Botnet	99.82	99.81	99.79	99.86	99.83
DoS	99.86	99.85	99.84	99.89	99.87
Password	99.73	99.72	99.70	99.77	99.75
Probe	99.83	99.82	99.81	99.87	99.85
DDoS	99.91	99.90	99.89	99.94	99.92
U2R	99.17	99.16	99.14	99.19	99.16
Normal	100	100	100	100	100

Table 3
Comparative analysis on the InSDN dataset (%).

Methods	Specificity ↑	F1-score ↑	Sensitivity ↑	Accuracy ↑	Precision ↑
SATIDS [33]	-	98.45	98.50	99.39	98.40
OML ensemble model [34]	-	98.27	97.93	98.20	97.51
MLP [35]	-	97.30	97.10	98.00	97.70
RF-LSTM [36]	-	98.00	99.00	99.00	99.00
CNN [37]	-	98.72	98.98	98.63	98.45
proposed	99.71	99.70	99.68	99.74	99.72

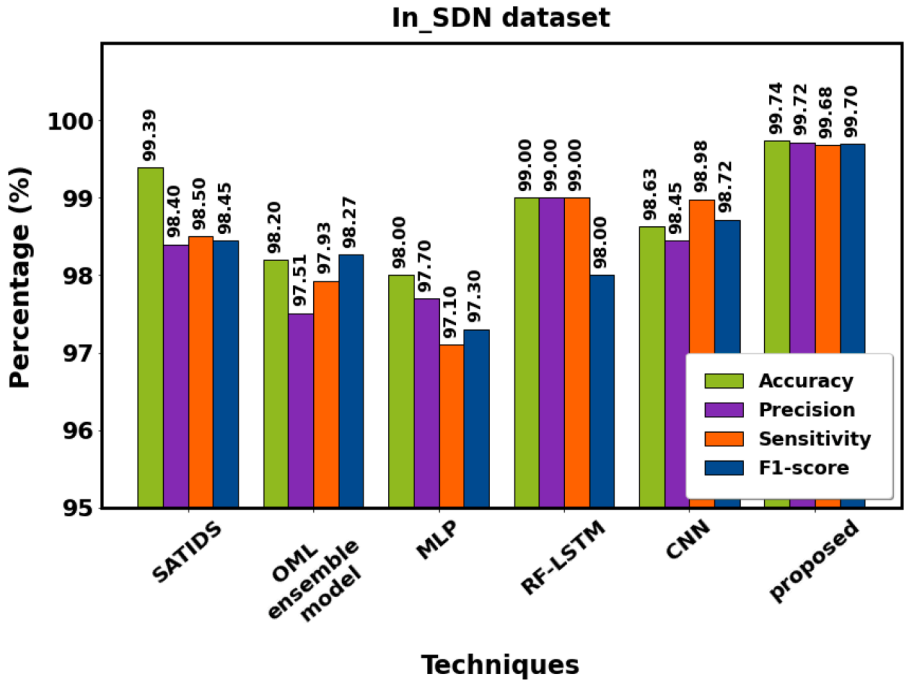


Fig. 5. Proposed approach's comparison with existing InSDN dataset.

sensitivity, accuracy and precision, it obtained 99.54 %, 99.55 %, 99.53 %, 99.56 %, and 99.61 %. As illustrated in Fig. 8, the accuracy of the multiclass prediction models has been verified and contrasted since the main drivers of achieving an increased detection rate are OOA-based feature selection and CTGAN-based data augmentation.

In comparison with all other methods, SVM performs poorly. For the Bot-IoT dataset, its accuracy is limited to 94.05 %. Scalability issues can arise with SVMs, particularly in large-scale SDN systems with considerable network traffic volume. Performance bottlenecks could result from the algorithm's growing time and memory needs. Like SVMs, CatBoost can use much memory during inference and training, mainly when working with deep trees and high-dimensional feature spaces. On SDN platforms with limited resources, this high memory utilization may restrict scalability and performance. However, in contrast to conventional deep learning architectures, CondenseNet employs a compact model structure in our suggested method, requiring fewer parameters. As a result, the model becomes lighter, which makes it better suited for installation on devices with few resources or in IDS systems with low processing power.

Fig. 9 shows the exact number of predictions for the different intrusion kinds for the BoT-IoT dataset classified using a confusion matrix. The suggested method correctly detects different sorts of benign attacks in this dataset, and its tp and tn rates are relatively high. On the other hand, the FlPv and FlNv rates (shown by different coloured squares) continue to be extremely low. The proportion of benign samples is significantly higher than that of other samples despite the absence of a colour square encircling the cases in those samples.

Apart from the confusion matrix, the ROC graph is a crucial tool for confirming the effectiveness of a security device. The percentage of accurately recognized positive events in deep learning is measured by a metric called True Positive Rate (TPR), which is sometimes referred to as sensitivity or recall. On the other hand, a result known as a True Negative Rate (TNR) occurs when the model accurately forecasts the adverse events. The ROC curve demonstrates a thoughtful examination of TPR and TNR; as a result, an IDS's effectiveness is actually assessed. The suggested method performs amazingly well on the ROC curve, as seen in Fig. 10.

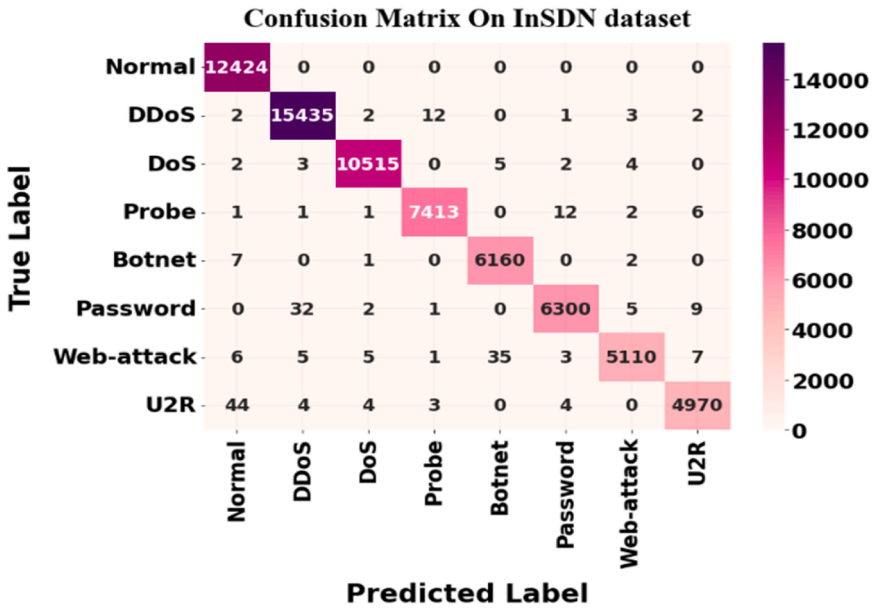


Fig. 6. Confusion matrix of InSDN dataset.

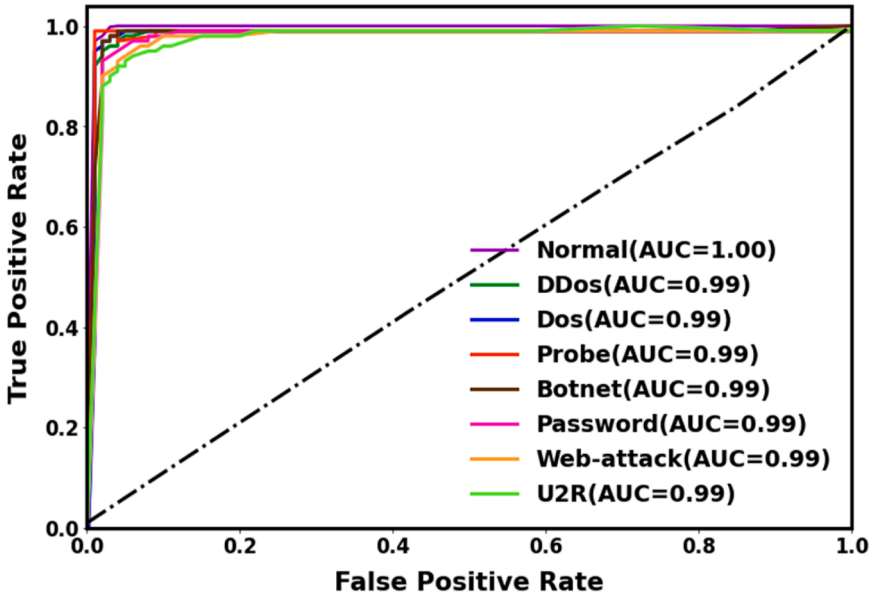


Fig. 7. ROC graph in InSDN dataset.

Table 4
Attack class comparison on the BoT-IoT dataset (%).

Categories	Specificity ↑	F1-score ↑	Sensitivity ↑	Accuracy ↑	Precision ↑
Dos	99.81	99.80	99.79	99.87	99.82
DDos	99.85	99.84	99.83	99.91	99.86
Theft	99.26	99.25	99.24	99.32	99.27
Reconnaissance	98.96	98.95	98.94	99.02	98.97
Normal	99.88	99.87	99.86	99.93	99.89

Table 5
Comparative analysis of the BoT-IoT dataset (%).

Methods	Specificity ↑	F1-score ↑	Sensitivity ↑	Accuracy ↑	Precision ↑
CNN [38]	-	-	-	95.55	-
Multigraph Neural Network [39]	-	97.74	96.63	97.74	97.79
Catboost [40]	-	96.50	94.70	96.64	98.50
KNN [41]	-	99.00	98.80	99.00	97.80
SVM [42]	-	94.12	94.05	94.05	94.66
proposed	99.55	99.54	99.53	99.61	99.56

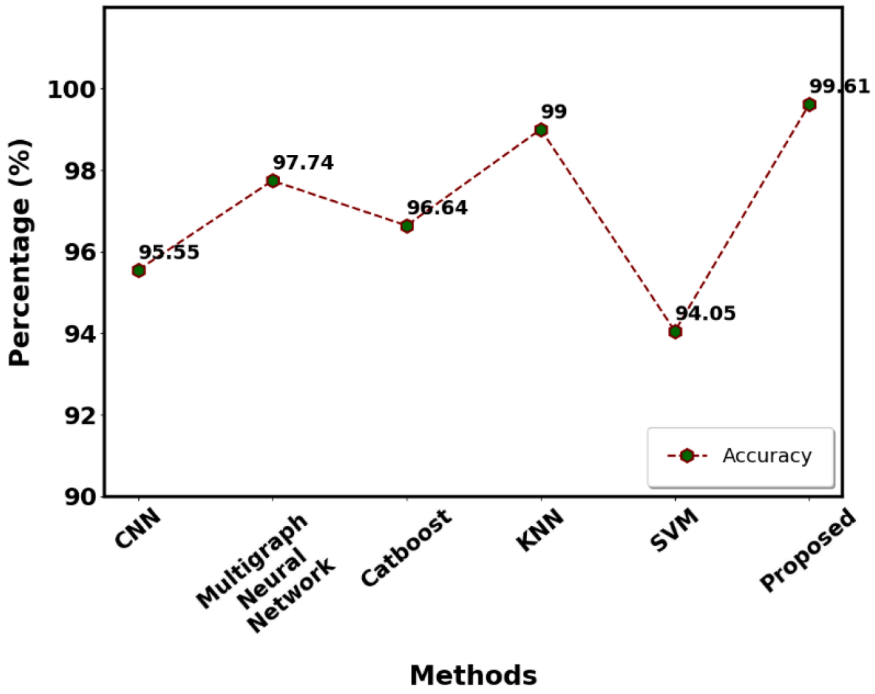


Fig. 8. Accuracy comparison on the BoT-IoT dataset.

4.5. Performance assessment on IoT-23 dataset

This section outlines and clarifies the findings from the hybrid model experiments on the IoT-23 dataset. Various performance indicators and representations have been employed to evaluate the method's effectiveness. In IoT device security, we conducted a critical study on the IoT-23 dataset using a hybrid approach that included CondenseNet and CoAtNet. This heterogeneous dataset, which includes a wide range of network traffic data, presents unique difficulties for attack identification. Table 6 shows the results of our model's classification of network traffic into distinct groups, including regular traffic and different kinds of attacks. Our research provided a thorough understanding of the model's functionality, emphasizing important parameters and assessment.

We have further evaluated the suggested model with current classifiers to demonstrate its effectiveness. Table 7 and Fig. 11 compare these models that were trained on the same dataset. The suggested method attained 99.64 % accuracy, 99.60 % precision, 99.58 % sensitivity, 99.57 % specificity, and 99.59 % F1 score, according to the table. This outcome is attained by the suggested CondesenNet-based approach's efficient feature extraction capacity. With its condensed and transition layers, it uses a hierarchical feature abstraction technique. The model can gradually extract and integrate information at various levels of abstraction thanks to its hierarchical structure, which also helps it identify local and global trends in the network traffic data.

The ETC-LSTM's performance is subpar when compared to all other techniques. Its accuracy was only 92 %. Because the LSTMs sequentially analyze input, their contextual comprehension is restricted to the current moment. This shortcoming may cause the model to fail to recognize subtle patterns or long-range relationships essential for successful intrusion detection in complicated network traffic data. Furthermore, with 96.3 % accuracy, the decision tree technique performs better. However, when it comes to expressing intricate links and interactions between features, decision trees are not very expressive. They might find it difficult to accurately represent nonlinear decision boundaries, particularly in complex or high-dimensional datasets.

As seen in Fig. 12, the confusion matrix offers an additional perspective on the model's categorization abilities. High scores along the diagonal demonstrate the model's strong classification performance, indicating accurate predictions. Nonetheless, some notable

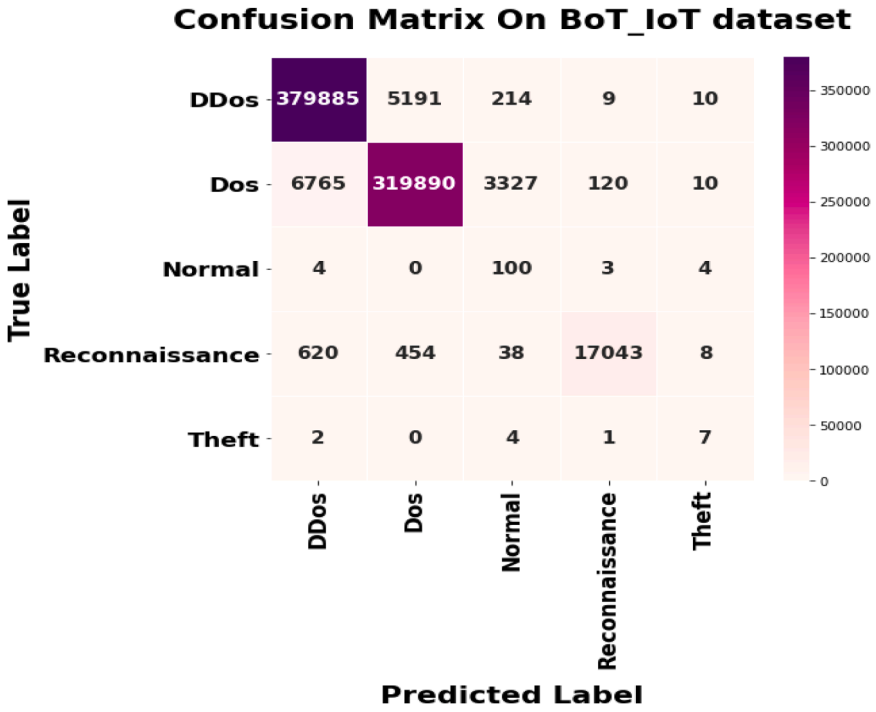


Fig. 9. Confusion matrix of Bot-IoT dataset.

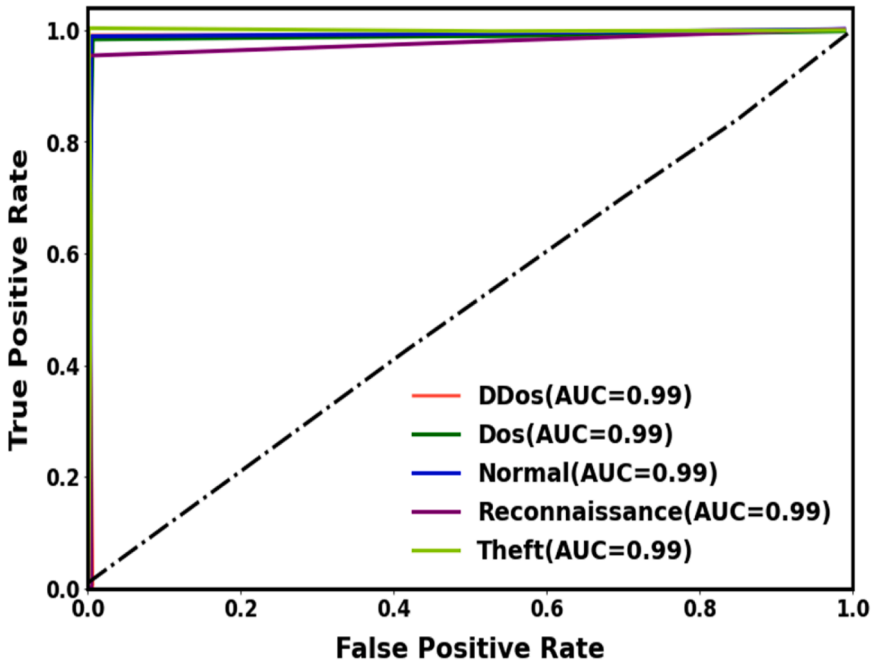


Fig. 10. ROC graph on Bot-IoT dataset.

off-diagonal features point to misclassifications. Mainly, there is a misclassification between the classes "heartbeat" and "Mirai" that calls attention and may indicate places where the model has to be improved. The similarity between the characteristics of these classes could cause these misclassifications. However, the presented method accurately categorizes most of the samples, indicating its excellent classification ability.

Furthermore, the method's inclusive superiority is highlighted by the ROC Curve, which is displayed in Fig. 13. Most of the classes'

Table 6
Attack class comparison on IoT-23 dataset (%).

Classes	Specificity ↑	F1-score ↑	Sensitivity ↑	Accuracy↑	Precision ↑
Command and control	99.22	99.24	99.23	99.29	99.25
Okiru	99.66	99.68	99.67	99.73	99.69
Mirai	99.72	99.74	99.73	99.79	99.75
PartofHorizontalPortScan	99.27	99.29	99.28	99.34	99.30
Heartbeat	99.39	99.41	99.40	99.46	99.42
Torii	99.58	99.60	99.59	99.65	99.61
DDoS	99.78	99.80	99.79	99.84	99.80
FileDownload	99.68	99.70	99.69	99.75	99.71
Normal	99.86	99.88	99.87	99.92	99.89

Table 7
Comparative analysis on the IoT-23 dataset (%).

Methods	Specificity ↑	F1-score ↑	Sensitivity ↑	Accuracy ↑	Precision ↑
ETC-LSTM [43]	-	91.00	92.00	92.00	92.00
CNN-LSTM [44]	-	93.99	94.32	95.00	94.89
KNN-ADWIN [45]	-	95.64	96.77	98.65	94.53
LSTM [46]	94.86	95.86	96.42	95.68	95.31
Decision Tree [47]	-	94.50	96.30	96.30	92.70
Proposed	99.57	99.59	99.58	99.64	99.60

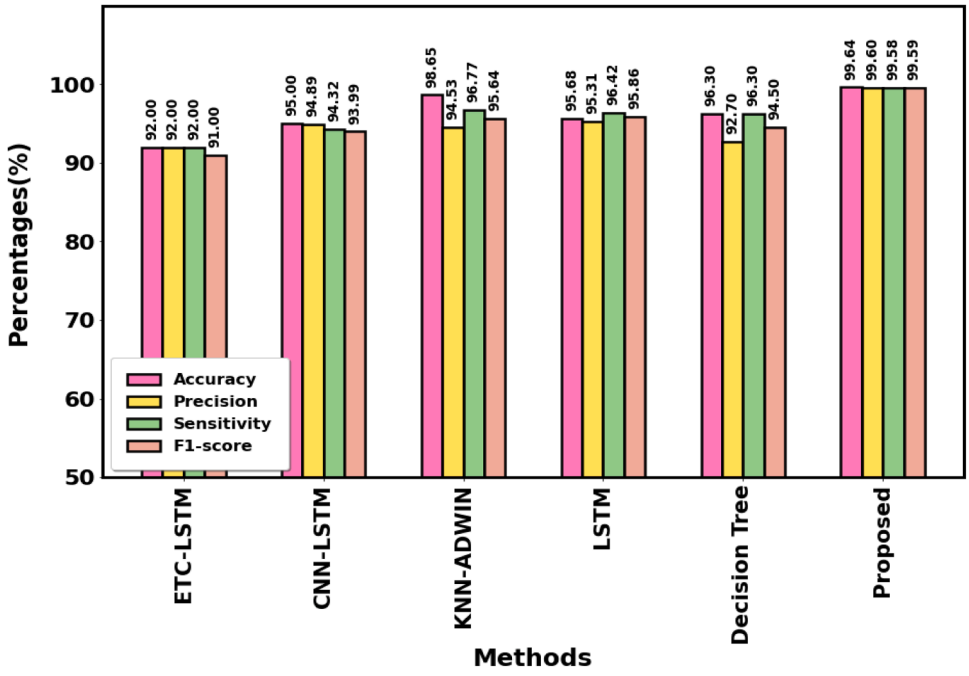


Fig. 11. Proposed approach's comparison on IoT-23 dataset.

trajectories, which are slowly moving towards the top-left quadrant of the graph, highlight the perfect balance between sensitivity and 1-specificity. The degree to which the model can differentiate among false positives and true positives is highlighted by the AUC for several classes that are close to the 1.0 threshold.

4.6. False alarm rate evaluation

This work also evaluates the FAR to improve evaluation. According to Table 8, the proposed approach's false alarm rate for the IoT-23, BoT-IoT, and InSDN datasets is just 0.0067, 0.0096, and 0.0091, respectively. With a lower false alarm rate, the suggested method is more likely to identify true positives with high accuracy and dependability while reducing false positives, which enhances overall efficacy and performance.

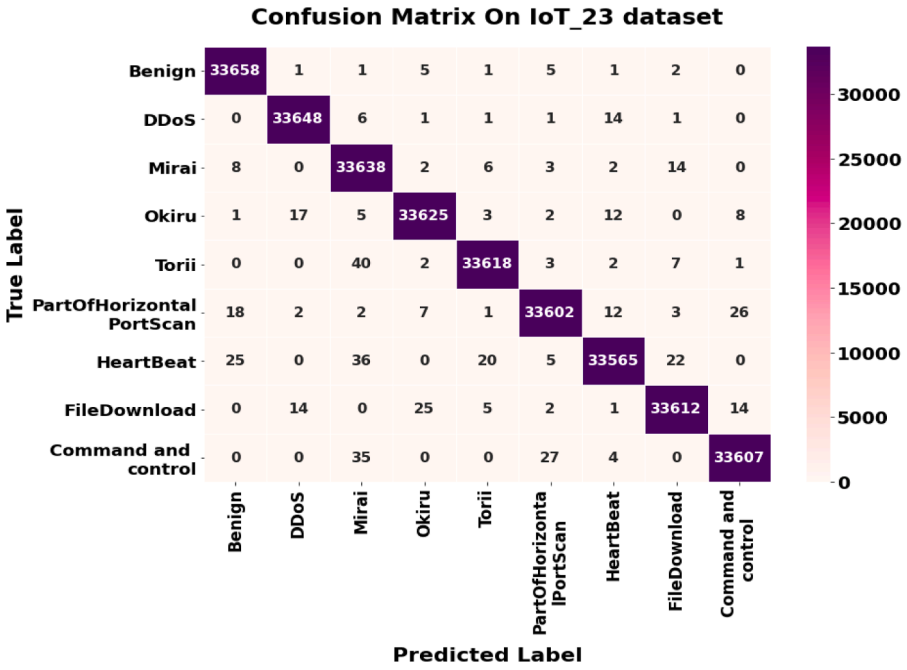


Fig. 12. Confusion matrix for the IoT-23 dataset.

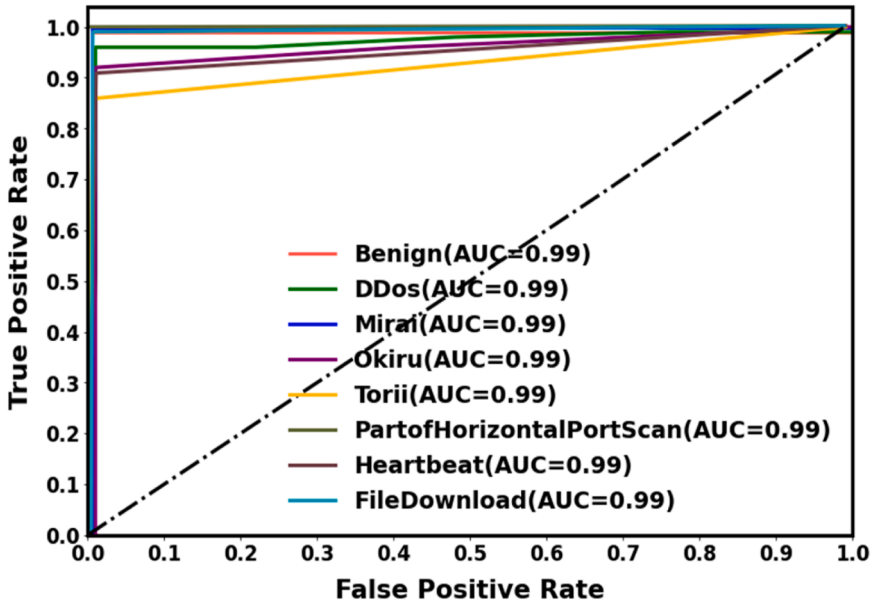


Fig. 13. ROC graph on Iot-23 dataset.

4.7. Effectiveness of feature selection

The suggested intrusion detection methodology is enhanced by applying an OOA method, which involves picking the most relevant features from the gathered features. The OOA-based technique reduces the number of features and delivers more information in the InSDN, BoT-IoT, and IoT-23 datasets. The performance of the suggested method with and without feature selection is shown in Table 9.

Table 9 demonstrates how the performance of the recommended strategy is improved by employing an effective OOA-based feature selection technique. It only obtains 99.21 %, 98.97 %, and 99.03 % accuracy for the InSDN, BoT-IoT, and IoT-23 datasets, correspondingly, without the feature selection method. The classifier performs better with the ideal set of features after the feature selection procedure and attains the best outcomes without using the feature selection technique.

Table 8
False alarm rate evaluation (%).

Technique	False Alarm Rate ↓
InSDN dataset	
CNN [48]	0.0300
MLP [35]	0.0400
Proposed	0.0067
Bot-IoT dataset	
Multigraph Neural network [39]	0.0109
SVM [49]	0.1480
proposed	0.0096
IoT-23 dataset	
Random forest [50]	0.8920
Decision tree [51]	0.0395
proposed	0.0091

Table 9
Impact of feature selection in the proposed approach (%).

Dataset	Accuracy ↑
Without feature selection	
InSDN	99.21
BoT-IoT	98.97
IoT-23	99.03
With feature selection	
InSDN	99.74
BoT-IoT	99.61
IoT-23	99.64

4.8. Mitigation result

Analyzing our suggested system's mitigation method is crucial. Fig. 14 displays our system's CPU utilization throughout the attack and subsequent remediation. The CPU is used to its maximum capacity during an attack, which raises latency. The graphic indicates that when the mitigation module of the suggested solution is called, the attack is mitigated in 3–4 s. Network traffic increases significantly during the attack, but it returns to normal after the mitigation mechanism is triggered. Following mitigation, the controller's efficiency becomes stable at roughly 28 %, while the server operates at roughly 60 % on average.

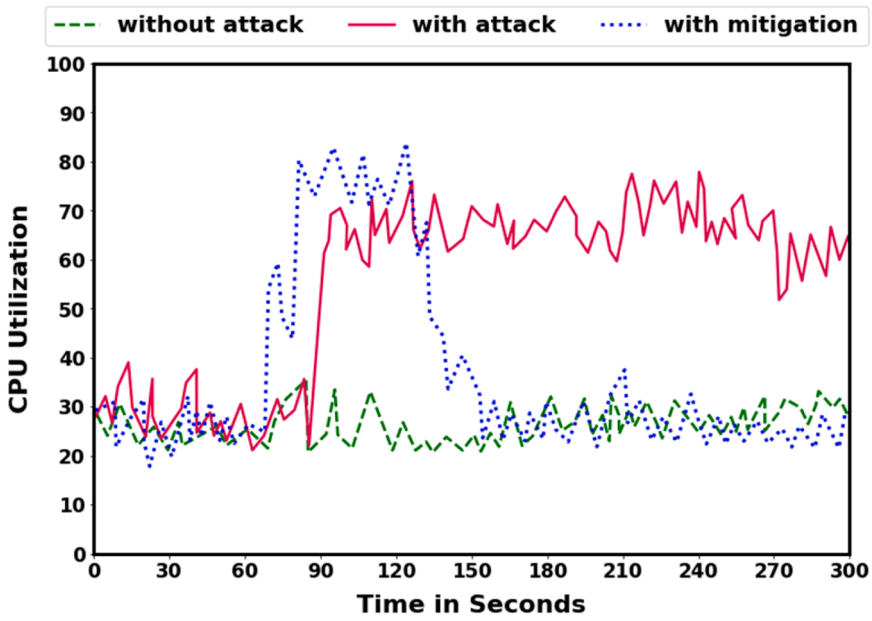


Fig. 14. CPU utilization comparison with and without mitigation.

Furthermore, Table 10 presents mitigation time, packet loss, and average delay analysis, all of which are essential to evaluating our mitigation method. In the absence of the mitigation process, latency spikes as a result of poorly optimized task handling and network overload, and packet loss increases as a result of congestion and the incapacity to address attacks effectively. As a result of the mitigation procedure, packet loss is reduced to almost zero, and the network performs at its best, which significantly lowers latency. It demonstrates that the recommended mitigation approach effectively and quickly lowers the intrusions. This technique lowers packet loss and average latency as well. Therefore, the proposed defence system effectively lowers intrusions and enhances the security of the network. Finally, the comprehensive results show that the recommended method is the most successful in locating and resolving attacks in the SDN-IoT environment. Additionally, the recommended intrusion detection model can precisely process the massive data created in the SDN-IoT framework.

4.9. Statistical analysis

To confirm the importance of the obtained data, statistical analysis was carried out. The ANOVA method has been used to validate the findings and the results are presented in Table 11. The ANOVA test findings show that the means of the three datasets under comparison differ statistically significantly from one another. There is a significant difference between the techniques, as indicated by the F statistic of 5.7552, which shows that the variability between the groups is significantly higher than the variability within the groups. The observed changes are unlikely to have happened by chance, as indicated by the p-value of 0.01499, which is below the frequently accepted significance level of 0.05. These findings allow for the rejection of the null hypothesis, which holds that all group means are equal. As a result, we draw the conclusion that proposed approach differs greatly from the others.

4.10. Computational cost analysis

We also examined the computing cost of the proposed approach. Table 12 illustrates how the framework's computing cost is determined by memory utilization, parameters, and floating point operations (FLOPs). It is evident from the table that CondenseNet and CoatNet have suitable computing costs for intrusion detection. At a marginally higher computational cost, the CoAtNet provides greater flexibility and model richness, whereas the CondenseNet is more effective with fewer FLOPs and memory use. The deployment of CoAtNet is justified by its ability to recognize threats with high reliability. These models may effectively predict intrusion detection in real-world scenarios while preserving computing efficiency.

4.11. Scalability analysis

To further ensure the suggested approach's performance, we analyze its scalability, and the results are presented in Fig. 15. It shows how raising the size of the dataset enhances the proposed method's accuracy for three distinct datasets: InSDN, IoT-23 and BoT-IoT. When the quantity of a dataset expands from 10 % to 100 %, accuracy improves uniformly. It implies that the system can easily manage larger datasets and gets better with more data. If accuracy keeps increasing and stays steady without decreasing, it means that growing data is not ruining the framework's performance.

4.12. Discussion

Using three benchmark datasets, namely BoT-IoT, InSDN, and IoT-23, the suggested hybrid deep learning framework is validated. Across all three datasets, the experimental result demonstrates a detection accuracy of over 99.50 %. We could conclude from the investigational analysis that the presented model performs superior than other competent models. Furthermore, based on our experimental study, we found that our approach produces the lowest false alarm rate when compared to other approaches, demonstrating that it successfully reduces the amount of normal cases that are mistakenly categorized as intrusions. It implies that our method is quite effective at differentiating between malicious and genuine data, which minimizes unwanted interruptions and improves network performance. It also performed remarkably well in recognizing real positive cases (sensitivity) and accurately identifying non-attack instances (specificity), as seen by the fact that it obtained the best sensitivity and specificity across the three datasets. It illustrates the model's resilience in correctly differentiating between attack and non-attack traffic in a range of network conditions. The superior performance of our proposed method is due to the efficiency of CondenseNet-based feature extraction, which enables the model to collect the most relevant information while reducing computational complexity. Furthermore, using OOA feature selection ensures that only the most significant and impactful characteristics are used. When combined with CoAtNet-based classification, which uses a hybrid attention mechanism to increase attack detection accuracy and robustness, our method outperforms other approaches

Table 10
The evaluation result of mitigation.

Number of attacks	Without mitigation		With mitigation		
	Packet loss	Average latency	Packet loss	Average latency	Mitigation time (s)
1	2.5	0.79	0	0.45	1
3	3.4	0.86	0	0.53	2
5	4.1	0.96	0	0.62	3

Table 11
ANOVA test.

Source	DF	Sum of Square	Mean Square	F Statistic	P-value
Groups (between groups)	2	43.0306	21.5153	5.7552	0.01499
Error (within groups)	14	52.3373	3.7384		
Total	16	95.3679	5.9605		

Table 12
Computational analysis.

Methods	Parameters	Flops	Memory
CondenseNet	0.52M	55	35MB
CoAtNet	17M	86	85MB

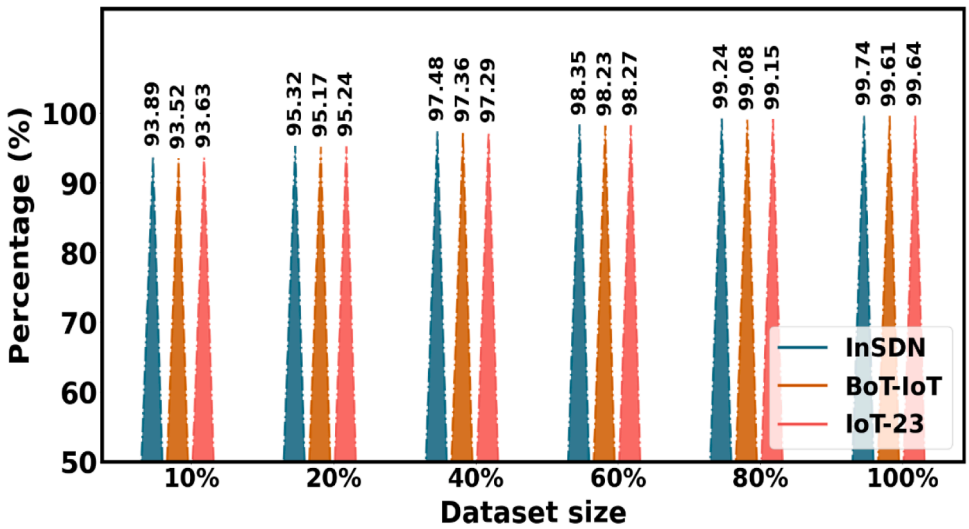


Fig. 15. Scalability analysis.

Table 13
Ablation study (%).

Preprocessing	CTGAN	CondenseNet	OOA	CoAtnet	Accuracy ↑	Precision ↑	Sensitivity ↑	Specificity ↑	F1-score ↑
InSDN dataset									
×	✔	✔	✔	✔	99.66	99.64	99.60	99.63	99.62
✔	×	✔	✔	✔	99.57	99.55	99.51	99.54	99.53
✔	✔	×	✔	✔	99.45	99.43	99.39	99.42	99.41
✔	✔	✔	×	✔	99.38	99.36	99.32	99.35	99.34
✔	✔	✔	✔	×	9 9.30	99.28	99.24	99.27	99.26
✔	✔	✔	✔	✔	**99.74**	**99.72**	**99.68**	**99.71**	**99.70**
BoT-IoT									
×	✔	✔	✔	✔	99.54	99.50	99.47	99.49	99.48
✔	×	✔	✔	✔	99.46	99.41	99.38	99.40	99.39
✔	✔	×	✔	✔	99.32	99.27	99.24	99.26	99.25
✔	✔	✔	×	✔	99.23	99.18	99.15	99.17	99.16
✔	✔	✔	✔	×	9 9.17	99.12	99.09	99.11	99.10
✔	✔	✔	✔	✔	**99.61**	**99.56**	**99.53**	**99.55**	**99.54**
IoT-23									
×	✔	✔	✔	✔	99.59	99.55	99.53	99.52	99.54
✔	×	✔	✔	✔	99.50	99.46	99.44	99.43	99.45
✔	✔	×	✔	✔	99.38	99.34	99.32	99.29	99.31
✔	✔	✔	×	✔	99.25	99.21	99.19	99.18	99.20
✔	✔	✔	✔	×	9 9.19	99.15	99.13	99.12	99.14
✔	✔	✔	✔	✔	**99.64**	**99.60**	**99.58**	**99.57**	**99.59**

with high sensitivity, specificity, and low false alarm rates. Furthermore, the CTGAN-based data augmentation is essential for resolving the dataset's imbalance. This technique ensures consistent performance across a variety of network traffic scenarios by improving the model's ability to identify uncommon and developing attack patterns through the generation of realistic synthetic attack samples. Furthermore, the suggested mitigation technique actively counteracts the attacks, guaranteeing better response time and reducing the potential impact they could bring. Network security and dependability are improved as a result, particularly in SDN environments where prompt action is essential. All of these elements have worked together to provide our system with a higher precision, accuracy, f1-score, sensitivity, specificity, and low false alarm rate than previous methods.

4.13. Ablation study

This section includes ablation studies that illustrate the effect of our proposed method. There are five modules in the suggested framework: preprocessing, CondenseNet-based feature extraction, OOA-based feature selection, CTGAN-based data augmentation, and CoAt-based classification. The testing results indicated above showed that the proposed method could produce positive results on the IoT-23, InSDN, and BoT-IoT datasets. The ablation experiment was further carried out in this section to examine the impacts of each element in the suggested framework, and the results are given in Table 13.

Each component's crucial roles in the suggested intrusion categorization framework are highlighted via the ablation study. The best performance is obtained when all of the components—Preprocessing, CTGAN, CondenseNet, OOA, and CoAtNet—are used together, proving their combined effectiveness. With an accuracy of 99.74 %, 99.61 %, and 99.64 % on the InSDN, BoT-IoT, and IoT-23 datasets, respectively, the results demonstrate that incorporating all components produces the best performance, demonstrating their collaborating effect. Any component that is removed causes performance to deteriorate; the effects of removing specific components, such as OOA and CoAtNet, are particularly noticeable. This research highlights the significance of each module and confirms the framework's design decisions, demonstrating how they work together to achieve robust and trustworthy intrusion categorization.

5. Current limitations and future scope

Some problems still need to be fixed even if the framework we have proposed is effective in recognizing and categorizing different kinds of intrusions. First, using virtual simulation, we trained and assessed the suggested model offline without building any real SDN networks. However, it's important to recognize online attacks and understand how this intrusion detection system can react quickly to an incident. In the future, we intend to test our proposed model in an actual setting. Additionally, we test the framework's performance with a single SDN controller; however, the latency and throughput vary among controllers. To guarantee equal understanding and show how well this method works with different controllers, future studies should concentrate on a wide range of controllers. Furthermore, the methodology presented only considers the SDN framework using three datasets. To expand the method's possible influence beyond the original context, future research will use more recent datasets to examine its implementation across a range of regions, including Fog, Edge, and Cloud. Although our suggested approach exhibits excellent computing efficiency, scalability, and accuracy, we did not directly assess energy, latency and power usage. These factors have a significant influence on the system's usability and real-world adoption in SDN contexts with resource-constrained IoT devices. In subsequent research, we will expand our investigation to evaluate the effect of the IDS on low-power IoT systems by including the measurements of energy usage, processing efficiency and latency.

6. Conclusion and future work

Identification of network attacks is necessary for SDN-IoT security to monitor and limit unwanted traffic flows. Thus, the main goal of this research is to develop an IDS based on a CondenseNet-CoAtNet methodology. Furthermore, a lightweight explainable intelligent display system (IDS) with optimal features is proposed using an efficient OOA-based feature selection technique. This comprehensive approach seeks to address the main issues with SDN-based intrusion detection, such as increased accuracy, managing issues with unbalanced data, and guaranteeing adaptability and scalability in dynamic network environments. The proposed method is evaluated using comprehensive benchmarking datasets such as InSDN, BoT-IoT, and IoT-23. The results show how effective the recommended method is, surpassing even the most sophisticated intrusion detection systems now in use. The algorithm notably shows an impressive accuracy rate of 99.64 %, 99.61 %, and 99.74 % on the IoT-23, BoT-IoT, and InSDN datasets, respectively. Moreover, the suggested mitigation approach decreases the effect of identified threats by shutting source ports, maintaining the steadiness of the network. Also, the suggested mitigation flow rules effectively manage the network, avoiding interruptions imposed by attacks. Therefore, the performance impact on switches is also negligible. It guarantees the network runs smoothly and safeguards against interruptions caused by attacks. In future, the system's overhead and resilience will be assessed in an actual SDN-IoT context.

Ethics approval

This material is the author's original work, which has yet to be previously published elsewhere. The paper reflects the author's research and analysis truthfully and completely.

CRediT authorship contribution statement

Dimmiti Srinivasa Rao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Resources, Writing – original draft, Writing – review & editing, Visualization. **Ajith Jubilson Emerson:** Conceptualization, Writing – review & editing, Writing – original draft, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

We declare that this manuscript is original, has not been published before, and is not currently being considered for publication elsewhere.

Appendix-A

Newly generated dataset samples for three datasets

InSDN dataset					
Types of Attack	Initial samples	After eliminating duplicate records	Training set samples	Newly generated samples	Total number of samples in training set after augmentation
Normal	68,424	–	54,739	–	54,739
DDoS	121,942	–	97,553	Certain instances are removed to keep the dataset balance.	54,739
DoS	53,616	–	42,892	11,847	54,739
Probe	98,129	–	78,503	Certain instances are removed to keep the dataset balance.	54,739
Botnet	164	–	131	54,608	54,739
Password	1405	–	1124	53,615	54,739
Web-attack	192	–	153	54,586	54,739
U2R	17	–	13	54,726	54,739
IoT-23 dataset					
Benign	30,858,735	2113,860	,691,088	Certain instances are removed to keep the dataset balance.	54,739
DDoS	19,538,713	3643,225	,914,580	Certain instances are removed to keep the dataset balance.	54,739
Mirai	9400	9400	7520	47,219	54,739
Okiru	60,990,711	234,942	187,953	Certain instances are removed to keep the dataset balance.	54,739
Torii	30	30	24	54,715	54,739
PartofHorizontalPortScan	213,853,817	369,525	295,620	Certain instances are removed to keep the dataset balance.	54,739
Heart beat	34,518	22,982	18,385	36,354	54,739
FileDownload	71	71	56	54,683	54,739
Command and control	21,995	18,939	15,151	39,588	54,739
Bot-IoT dataset					
Dos	33,005,194	–	26,404,155	Certain instances are removed to keep the dataset balance.	54,739
DDos	38,532,480	–	30,825,984	Certain instances are removed to keep the dataset balance.	54,739
Theft	1587	–	1270	53,469	54,739
Reconnaissance	4,70,655	–	376,524	Certain instances are removed to keep the dataset balance.	54,739
Normal	9543	–	7634	47,105	54,739

Appendix-B

List of selected features

Dataset	Number of selected features	Selected features
BoT-IoT	10	Seq, stddev, NINConn PDstIP, Min, Max, Srate, Drate, pKSeqID, NINConn PSrcIP, Mean,
InSDN	20	Dst Port, Init Bwd Win Byts, Pkt Len Max, Bwd Header Len, Bwd Pkt Len Max, PSH Flag Cnt, Src Port, Flow IAT Std, Flow IAT Max, Active Min, Idle Min, Subflow Fwd Pkts, Bwd Pkt Len Mean, Subflow Fwd Pkts, Flow IAT Mean, Fwd Act Data Pkts, Down/Up Ratio, Protocol, RST Flag Cnt, Tot Fwd Pkts
IoT-23	10	Label, conn_state, id.orig_h, resp_ip_bytes, orig_ip_bytes, orig_pkts, duration, id.res_p, id.orig_p, ts

Data availability

Data will be made available on request.

References

[1] Arthi R, Krishnaveni S, Zeadally S. An intelligent SDN-IoT enabled intrusion detection system for healthcare systems using a hybrid deep learning and machine learning approach. China Communications; 2024.
[2] Rui K, Pan H, Shu S. Secure routing in the Internet of Things (IoT) with intrusion detection capability based on software-defined networking (SDN) and Machine Learning techniques. Sci Rep 2023;13(1):18003.
[3] Shaji NS, Muthalagu R, Pawar PM. SD-IIDS: intelligent intrusion detection system for software-defined networks. Multimed Tools Appl 2024;83(4):11077–109.
[4] Alzahrani AO, Alenazi MJ. ML-IDSDN: machine learning based intrusion detection system for software-defined network. Concurr Comput Pract Exp 2023;35(1):e7438.
[5] Kumar C, Biswas S, Ansari MSA, Govil MC. Nature-inspired intrusion detection system for protecting software-defined networks controller. Comput Secur 2023;134:103438.
[6] Chetouane A, Karoui K. Risk based intrusion detection system in software defined networking. Concurr Comput Pract Exp 2024;36(9):e7988.
[7] Qiu F, Xu H, Li F. Applying modified golden jackal optimization to intrusion detection for software-defined networking. Electr Res Arch 2024;32(1):418–44.
[8] Revathi M, Devi SK. Hybrid architecture for mitigating DDoS and other intrusions in SDN-IoT using MHDBN-W deep learning model. Int J Mach Learn Cybern 2024:1–22.
[9] Zhang L, Liu K, Xie X, Bai W, Wu B, Dong P. A data-driven network intrusion detection system using feature selection and deep learning. J Inf Secur Appl 2023;78:103606.
[10] Safwan H, Iqbal Z, Amin R, Khan MA, Alhaisoni M, Alqahtani A, Chang B. An IoT environment-based framework for intelligent intrusion detection. CMC Comput. Mater. Contin 2023;75:2365–81.
[11] Deveci A, Yilmaz S, Sen S. Evolving lightweight intrusion detection systems for RPL-based Internet of Things. In: Proceedings of the international conference on the applications of evolutionary computation (Part of evostar). Cham: Springer Nature Switzerland; 2023. p. 177–93.
[12] Zainudin A, Akter R, Kim DS, Lee JM. Federated learning inspired low-complexity intrusion detection and classification technique for SDN-based industrial CPS. IEEE Trans Netw Serv Manag 2023;20(3):2442–59.
[13] Jafarian T, Ghaffari A, Seyfollahi A. Detecting and mitigating security anomalies in software-defined networking (SDN) using gradient-boosted trees and floodlight controller characteristics. Comput Stand Interfaces 2024;91:103871.
[14] Malini P, Kavitha KR. An efficient deep learning mechanisms for IoT/non-IoT devices classification and attack detection in SDN-enabled smart environment. Comput Secur 2024;141:103818.
[15] Babbar H, Rani S. FRHIDS: federated learning recommender hydrid intrusion detection system model in software defined networking for consumer devices. IEEE Trans Consum Electr 2023;70(1):2492–9.
[16] Alshehri MS, Saidani O, Alrayes FS, Abbasi SF, Ahmad J. A self-attention-based deep convolutional neural networks for IIoT networks intrusion detection. IEEE Access; 2024.
[17] Jin Y, Xu H, Qin Z. Intrusion detection model for software-defined networking based on feature selection. In: Proceedings of the sixth international conference on computer information science and application technology (CISAT 2023). 12800. SPIE; 2023. p. 428–34.
[18] Charanarur P, Thanh Hung B, Chakrabarti P, Siva Shankar S. Design optimization-based software-defined networking scheme for detecting and preventing attacks. Multimed Tools Appl 2024;83:71151–69.
[19] Mustafa O, Ali K, Naqash T. C-RADAR: a centralized deep learning system for intrusion detection in software defined networks. In: Proceedings of the international conference on communication, computing and digital systems (C-CODE). IEEE; 2023. p. 1–6.
[20] Nawshin S, Islam S, Shatabda S. PCA-ANN: feature selection-based hybrid intrusion detection system in a software-defined network. J Intell Fuzzy Syst 2023;47:273–90 (Preprint).
[21] Songa AV, Karri GR. An integrated SDN framework for early detection of DDoS attacks in cloud computing. J Cloud Comput 2024;13(1):64.
[22] Mhamdi L, Isa MM. Securing SDN: hybrid autoencoder-random forest for intrusion detection and attack mitigation. J Netw Comput Appl 2024;225:103868.
[23] Wahab F, Shah A, Khan I, Ali B, Adnan M. An SDN-based hybrid-DL-driven cognitive intrusion detection system for IoT ecosystem. Comput Electr Eng 2024;119:109545.
[24] Alasali T, Dakkak O. A novel DDoS detection method using multi-layer stacking in SDN environment. Comput Electr Eng 2024;120:109769.
[25] Khedr WI, Gouda AE, Mohamed ER. FMDADM: a multilayer DDoS attack detection and mitigation framework using machine learning for stateful SDN-based IoT networks. IEEE Access 2023;11:28934–54.
[26] Bhayo J, Shah SA, Hameed S, Ahmed A, Nasir J, Draheim D. Towards a machine learning-based framework for DDoS attack detection in software-defined IoT (SD-IoT) networks. Eng Appl Artif Intell 2023;123:106432.
[27] Polat O, Türkoğlu M, Polat H, Oyucu S, Üzen H, Yardımcı F, Aksöz A. Multi-stage learning framework using convolutional neural network and decision tree-based classification for detection of DDoS pandemic attacks in SDN-based SCADA systems. Sensors 2024;24(3):1040.
[28] El-Sayed A, Said W, Tolba A, Alginahi Y, Toony AA. MP-GUARD: a novel multi-pronged intrusion detection and mitigation framework for scalable SD-IOT networks using cooperative monitoring, ensemble learning, and new P4-extracted feature set. Comput Electr Eng 2024;118:109484.
[29] Majidian SZ, TaghipourEivazi S, Arasteh B, Ghaffari A. Optimizing random forests to detect intrusion in the internet of things. Comput Electr Eng 2024;120:109860.

[30] Zavrak S, Iskefiyeli M. Flow-based intrusion detection on software-defined networks: a multivariate time series anomaly detection approach. Neural Comput Appl 2023;35(16):12175–93.
[31] Xu L, Skoularidou M, Cuesta-Infante A, Veeramachaneni K. Modeling tabular data using conditional gan. Adv Neural Inf Process Syst 2019;32:1–11.
[32] Dehghani M, Trojovský P. Osprey optimization algorithm: a new bio-inspired metaheuristic algorithm for solving engineering optimization problems. Front Mech Eng 2023;8:1126450.
[33] Elsayed RA, Hamada RA, Abdalla MI, Elsaid SA. Securing IoT and SDN systems using deep-learning based automatic intrusion detection. Ain Shams Eng J 2023; 14(10):102211.
[34] Alashhab AA, Zahid MS, Isyaku B, Elnour AA, Nagmeldin W, Abdelmaboud A. Abdullah T.A.A.,Maiwada U. Enhancing DDoS attack detection and mitigation in SDN using an ensemble online machine learning model. IEEE Access; 2024.
[35] Al Essa HA, Bhaya WS. Ensemble learning classifiers hybrid feature selection for enhancing performance of intrusion detection system. Bull Electr Eng Inform 2024;13(1):665–76.
[36] Safwan H, Iqbal Z, Amin R, Khan MA, Alhaisoni M, Alqahtani A, Kim YJ, Chang B. An IoT environment-based framework for intelligent intrusion detection. CMC Comput Mater Contin 2023;75:2365–81.
[37] Duong VD, Tran NK, Ta MT. Intrusion detection using network flow feature for software-defined networks. J Sci Tech Sect Inf Commun Technol 2023;12(02): 75–87.
[38] Saba T, Rehman A, Sadad T, Kolivand H, Bahaj SA. Anomaly-based intrusion detection system for IoT networks through deep learning model. Comput Electr Eng 2022;99:107810.
[39] Altaf T, Wang X, Ni W, Yu G, Liu RP, Braun R. A new concatenated multigraph neural network for IoT intrusion detection. Internet Things 2023;22:100818.
[40] Maurya S, Kumar S, Garg U, Kumar M. An efficient framework for detection and classification of IoT botnet traffic. ECS Sens Plus 2022;1(2):026401.
[41] Tyagi H, Kumar R. Attack and anomaly detection in IoT networks using supervised machine learning approaches. Rev d'Intell Artif 2021;35(1):11–21.
[42] Almaraz-Rivera JG, Perez-Diaz JA, Cantoral-Ceballos JA. Transport and application layer DDoS attacks detection to IoT devices by using machine learning and deep learning models. Sensors 2022;22(9):3367.
[43] Khanday SA, Fatima H, Rakesh N. A novel data preprocessing model for lightweight sensory IoT intrusion detection. Int J Math Eng Manag Sci 2024;9:188–204.
[44] Nazir A, He J, Zhu N, Qureshi SS, Qureshi SU, Ullah F, Wajahat A, Pathan MS. A deep learning-based novel hybrid CNN-LSTM architecture for efficient detection of threats in the IoT ecosystem. Ain Shams Eng J 2024;15(7):102777.
[45] Beshah YK, Abebe SL, Melaku HM. Drift adaptive online DDoS attack detection framework for IoT system. Electronics (Basel) 2024;13(6):1004.
[46] Sahu AK, Sharma S, Tanveer M, Raja R. Internet of Things attack detection using hybrid deep learning model. Comput Commun 2021;176:146–54.
[47] Thamaraiselvi D, Mary S. Attack and anomaly detection in IoT networks using machine learning. Int J Comput Sci Mob Comput 2020;9(10):95–103.
[48] Wang K, Fu Y, Duan X, Liu T, Xu J. Abnormal traffic detection system in SDN based on deep learning hybrid models. Comput Commun 2024;216:183–94.
[49] Khraisat A, Gondal I, Vamplew P, Kamruzzaman J, Alazab A. A novel ensemble of hybrid intrusion detection system for detecting internet of things attacks. Electronics (Basel) 2019;8(11):1210.
[50] Sarobin MVR, Ranjith J, Ashwath D, Vinithi K, Khushi V. Comparative analysis of various feature extraction methods on IoT 2023. Procedia Comput Sci 2024; 233:670–81.
[51] Jayaraman B, Thai MTNT, Anand A, Anandan KR. Detecting malicious IoT traffic using Machine learning techniques. Roman J Inf Technol Autom Control 2023; 33(4):47–58.

Dimmiti Srinivasa Rao is a Ph.D. Research scholar at VIT-AP University, specializing in security in software development, machine learning, DevOps, and deep learning techniques. Her research focuses on developing algorithms for preserving security in testing and deployment. He has published some papers in appropriate conferences and journals.

Dr. **Ajith Jubilson Emerson** is working as an Associate Professor of Computer Science and Engineering at Vellore Institute of Technology, Andhra Pradesh. He completed his Ph.D. from VIT-AP University in 2020. He has teaching experience of 10 years, guided over 100 students on independent research projects over the last 8 years, and personally supervised 10+ undergraduate research interns in the last 5 years. He has >10 papers in indexed journals in the domain of network security, data analytics, and machine learning.